

Code Repo Knowledge Layer

Code Repo Knowledge Layer

팀 간 커뮤니케이션이 부족한 코드 레포에 지식 레이어를 얹는 아이디어.

- 핵심 문제 정리: 코드 레포는 있는데 사람들이 소통을 안 하니까, 새로운 사람이 오면 코드를 직접 읽어야 하고, 각 파트가 뭘 하는지 파악하는 데 시간이 너무 많이 걸림.

- 브레인스토밍 방향들

1. AI Coaching Agent — "Repo Onboarding Buddy"

새로 온 사람이 "이 레포에서 데이터 파이프라인은 어떻게 돌아가?" 하고 물어보면 코드 구조, 의존성, 히스토리를 분석해서 대답해주는 에이전트. 단순 문서 검색이 아니라 코드 자체를 이해하고 설명해주는 것.

확장 아이디어:

질문 로그 → 자동 **FAQ** 생성: 새 사람들이 반복적으로 묻는 질문을 트래킹해서 자동으로 "자주 묻는 질문" 문서를 만들어냄

"이 함수를 왜 이렇게 짰어?" 히스토리 추적: **git blame + commit message + PR discussion**을 종합해서 의사결정 맥락을 복원

난이도별 설명: 주니어에게는 개념 중심, 시니어에게는 아키텍처 중심으로 설명 톤 조절

2. Auto-Documentation — 단순 생성을 넘어서 자동 문서화를 "creative process"로 본 건 좋은 관점인데, 여기서 더 밀어붙일 수 있는 것들:

Living Documentation: 코드가 바뀔 때마다 문서가 자동 업데이트. **stale docs** 문제 해결

다층 문서화: 같은 코드에 대해 (1) 한 줄 요약 (2) 기능 설명 (3) 아키텍처 맥락 (4) 비즈니스 임팩트 — 4개 레이어를 자동 생성

코드 변경 → 문서 **diff**: **PR** 올리면 "이 변경으로 인해 문서에서 이 부분이 업데이트되어야 합니다" 자동 제안

비공식 지식 캡처: **Slack**에서 "아 그거 이렇게 하면 돼" 같은 대화를 감지해서 관련 코드 문서에 자동 링크

3. Usage Tracking Layer — "어떻게 쓰이고 있는가"

데이터 사이언스 관점에서 가장 흥미로운 부분:

코드 사용 패턴 분석: 어떤 모듈이 가장 많이 **import** 되는지, 어떤 함수가 실제로는 안 쓰이는지

의존성 임팩트 맵: "이 함수 바꾸면 누구한테 영향 가지?" 를 실시간으로 보여주는 대시보드

팀별 ownership 자동 감지: git 히스토리 기반으로 "이 모듈은 사실상 A팀이 관리하고 있다"를 자동 매핑

Knowledge bus factor 경고: "이 코드를 이해하는 사람이 1명뿐입니다" 알림

4. Interactive Code Map

정적 문서 대신 탐색 가능한 인터랙티브 맵:

코드 레포를 시각적 그래프로 보여주고, 클릭하면 점점 깊이 들어가는 구조
"이 데이터는 어디서 와서 어디로 가는가" **data lineage**를 시각화
팀 간 코드 의존성을 보여줘서 "우리가 모르는 사이에 B팀 코드에 의존하고 있었다"를 발견

5. 커뮤니케이션 갭을 메우는 방향

근본 문제가 소통부재라면:

자동 변경 요약 뉴스레터: 매주 "이번 주에 레포에서 뭐가 바뀌었는지" AI가 요약해서 팀 전체에 발송

Cross-team impact alerts: A팀이 코드를 바꿨는데 B팀 파이프라인에 영향 가능성이 있으면 자동 알림

"Ask the repo" Slack bot: 아무 채널에서나 질문하면 레포 기반으로 답변

제품화한다면 — 가능한 MVP 순서

Phase 1: 코드 레포 연결 → AI Q&A 챗봇 (coaching agent)

Phase 2: 자동 문서 생성 + PR 연동

Phase 3: 사용 패턴 트래킹 대시보드

Phase 4: 팀 간 의존성/임팩트 분석

Usage Tracking Layer (Simple Road Map)

Phase 1: Data Source Identification and Definition of Criteria

The first step is to establish Git history as the primary data source. Before conducting any analysis, it is critical to clearly define what data will be collected and how “unused code” will be operationally defined.

The classification of unused code will be determined based on a combination of the following criteria: Modification frequency / Recency of the last modification / Cross-file reference frequency

If these criteria are not clearly defined, the credibility of the overall analysis may be compromised. Therefore, the definition stage is considered the most critical foundation of the project.

Phase 2: Data Extraction

Git history will be extracted using the Python library **PyDriller**, and transformed into structured datasets in the form of Pandas DataFrames.

The extracted data will include: File name / Modification date / Author / Lines added / Lines deleted / Commit frequency

These metrics will enable quantitative analysis of code evolution and modification patterns over time.

Phase 3: Static Analysis for Usage Detection

To determine whether code is actively used, static analysis will be conducted using **Abstract Syntax Tree (AST) parsing** or tools such as the **vulture library** to identify unused functions and classes.

Relying solely on modification history is insufficient. A meaningful assessment requires verifying whether functions or classes are actually referenced within the codebase. This step ensures that the analysis reflects practical usage rather than surface-level activity.

Phase 4: Scoring and Classification Framework

A scoring system will be designed to classify code into three categories:

1. Core Assets
Code that is frequently modified and widely referenced across files.
These components represent critical organizational assets.
2. Stable (Managed Components)
Code with low modification frequency but high cross-file reference usage, such as utility modules.
These components require stability and proper documentation rather than frequent updates.
3. Dead Code Candidates
Code with low modification frequency and minimal or no references.
These components may be considered for repository cleanup, archival, or deprecation.

This classification framework is intended not merely as a statistical summary, but as a decision-support tool for structured technical asset management.

Phase 5: Dashboard Visualization

The results will be visualized using **Tableau**, with the repository structure represented in a Treemap format.

1. Low-usage components will be displayed in darker or muted colors.
2. Core Assets will be highlighted in stronger, more visible tones.

This visualization approach is designed to make repository health and asset distribution intuitively understandable—not only for engineers, but also for HR and management stakeholders.

프로젝트 AI 요약본

제품 한 줄 정의

"에디터를 떠나지 않고 레포의 모든 맥락을 이해하는 **VS Code Extension**"

개발자가 새 레포에 투입됐을 때, 또는 익숙한 레포에서도 다른 팀 코드를 건드려야 할 때, 지금은 이런 과정을 거칩니다: 코드를 열고 → 원지 모르겠으니 **Slack** 검색하고 → **Confluence** 뒤지고 → 결국 옆 사람한테 물어봅니다. 이 전체 과정을 에디터 안에서 해결하는 게 목표입니다.

풀고자 하는 진짜 문제

코드 레포에는 세 종류의 지식이 있습니다.

첫째, 명시적 지식. 코드 자체, 주석, **README**, **docstring**. 이건 이미 레포에 있습니다. 하지만 양이 많고 구조 파악이 어렵습니다.

둘째, 구조적 지식. "이 모듈이 저 모듈에 의존한다", "이 함수를 바꾸면 저쪽 서비스가 깨진다" 같은 관계 정보. 이건 코드에 암묵적으로 존재하지만, 사람이 머리로 추적하기엔 한계가 있습니다.

셋째, 맥락적 지식. "왜 이렇게 짰는지", "원래 다른 방식이었는데 성능 문제로 바꿨다", "이 모듈은 사실상 김 개발자만 이해한다" 같은 것들. 이건 사람들 머릿속에만 있고, 팀 간 소통이 없으면 완전히 유실됩니다.

기존 도구들은 첫째만 다룹니다. **GitHub Copilot**이 코드 자동완성을 해주고, 검색 도구가 코드를 찾아주지만, 둘째와 셋째는 여전히 사람에게 물어봐야 합니다. 이 프로젝트는 셋 다 커버하는 지식 레이어를 만드는 겁니다.

세 가지 핵심 기능의 컨셉

기능 **1**: 코드 **Q&A** — "물어보면 답해주는 레포 전문가"

기존 도구와의 차이점

일반적인 코드 **Q&A** 챗봇은 코드를 텍스트로 취급합니다. 파일 내용을 잘라서 **LLM**에 던지고 답을 받는 방식이죠. 이러면 "이 함수가 뭘 하는지"는 답할 수 있어도, "이 함수를 왜 이렇게 짰는지", "이 함수를 바꾸면 어디가 영향을 받는지"는 답할 수 없습니다.

이 도구의 **Q&A**는 세 가지 소스를 동시에 활용합니다.

코드 자체를 **AST**(추상 구문 트리) 수준에서 파싱해서 구조를 이해합니다. 단순히 텍스트 검색으로 관련 코드를 찾는 게 아니라, "이 함수가 호출하는 함수들", "이 클래스를 상속하는 클래스들" 같은 구조적 관계를 알고 있는 상태에서 답변합니다.

의존성 그래프를 참조합니다. 질문에 관련된 코드의 상위/하위 의존성을 함께 컨텍스트로 제공해서, **LLM**이 더 정확한 아키텍처 수준의 답변을 할 수 있게 합니다.

git 히스토리를 활용합니다. 해당 코드의 최근 변경 이력, 커밋 메시지, **PR** 디스커션을 함께 참조해서 "왜 이렇게 됐는지"까지 설명할 수 있습니다.

사용 시나리오 예시

개발자가 **auth/middleware.py**를 열고 있는 상태에서 사이드바 채팅에 "이 레포에서 인증 플로우가 어떻게 돌아가?"라고 묻습니다. 도구는 단순히 **auth** 폴더 파일들을 나열하는 게 아니라, 요청이 들어와서 인증을 거쳐 응답이 나가기까지의 전체 체인을 의존성 그래프에서 추적해서 설명합니다. "이 미들웨어가 먼저 토큰을 검증하고, 그 결과를 권한 체크 모듈로 넘기고, 최종적으로 라우터 핸들러에 도달합니다"처럼요.

여기서 "근데 왜 토큰 검증을 이렇게 두 단계로 나눴어?"라고 **follow-up**을 하면, **git** 히스토리에서 해당 코드가 분리된 커밋을 찾아서 "3개월 전 **PR #234**에서 성능 이슈로 분리됨. 디스커션에서 김 개발자가 '단일 검증이 병목이 돼서 비동기 분리했다'고 설명"이라는 맥락까지 제공합니다.

현재 파일 인식(**Context Awareness**)

중요한 점은, 사용자가 현재 어떤 파일을 열고 있는지를 에이전트가 알고 있다는 것입니다. 같은 "인증이 어떻게 돼?"라는 질문이라도 **auth/middleware.py**를 보고 있으면 미들웨어 중심으로, **api/routes.py**를 보고 있으면 라우트 핸들러 관점에서 설명합니다. 이게 일반 챗봇과의 결정적 차이입니다.

기능 2: 의존성/임팩트 분석 — "이거 바꾸면 어디가 깨지나"

왜 이게 필요한가

팀 간 소통이 없는 레포에서 가장 위험한 상황은, 한 팀이 자기 모듈을 수정했는데 다른 팀 코드가 조용히 깨지는 겁니다. 테스트가 완벽하면 **CI**에서 잡히겠지만, 현실적으로 모든 레포의 테스트 커버리지가 높지는 않습니다.

이 기능은 코드를 수정하기 전에 "이 변경의 영향 범위"를 보여주는 겁니다.

작동 방식

레포 전체의 코드를 파싱해서 의존성 그래프를 구축합니다. 노드는 함수, 클래스, 모듈이고, 엣지는 "호출한다", "**import** 한다", "상속한다" 같은 관계입니다.

사용자가 특정 함수나 클래스를 선택하고 "임팩트 분석"을 요청하면, 그래프에서 해당 노드를 시작점으로 역방향 탐색을 합니다. 이 함수를 직접 호출하는 곳(1차 영향), 그 호출자를 다시 호출하는 곳(2차 영향)을 단계별로 보여줍니다.

단순한 **grep**과의 차이

"이 함수 이름을 **grep** 하면 되는 거 아니야?"라고 생각할 수 있는데, 차이가 큼니다.

grep은 문자열 매칭이라 동명이인 문제가 있습니다. **utils.parse()**를 검색하면 완전히 다른 모듈의 **parse()**까지 잡힙니다. **AST** 기반 분석은 실제 **import** 관계를 추적하므로 정확합니다.

grep은 간접 영향을 못 봅니다. **A**가 **B**를 호출하고 **B**가 **C**를 호출할 때, **C**를 바꾸면 **A**도 영향받는데, **C**에서 **grep**해도 **A**는 안 나옵니다. 그래프 탐색은 이걸 잡아냅니다.

grep은 영향의 "심각도"를 모릅니다. 이 도구는 각 의존 경로에 대해 "직접 의존 vs 간접 의존", "테스트 존재 여부", "최근 변경 빈도" 같은 메타데이터를 함께 보여줘서 어디가 진짜 위험한지 판단할 수 있게 합니다.

시각화

VS Code 내에 웹뷰 패널로 의존성 그래프를 시각화합니다. 선택한 함수를 중심으로 영향받는 노드들이 퍼져나가는 그래프를 보여주고, 각 노드를 클릭하면 해당 코드로 점프합니다. 색상으로 심각도를 구분하고요 — 직접 의존은 빨간색, 간접 의존은 주황색, 테스트가 있는 건 초록 테두리.

기능 3: 자동 문서화 — "코드와 함께 숨 쉬는 문서"

기존 자동 문서화의 문제

JSDoc이나 **Sphinx** 같은 기존 도구는 "코드에서 문서를 생성"하지만, 생성 시점의 스냅샷일 뿐입니다. 코드가 바뀌면 문서는 **stale**해지고, 아무도 업데이트 안 합니다. 또한 이런 도구들은 **API** 시그니처 수준의 문서만 만들지, "이게 아키텍처에서 어떤 역할인지", "비즈니스적으로 왜 중요한지"는 안 알려줍니다.

다층 문서화 컨셉

이 도구는 같은 코드에 대해 네 가지 레벨의 문서를 생성합니다.

L1 — 한 줄 요약. 함수/클래스 위에 **Hover**했을 때 보이는 한 줄. "사용자 인증 토큰을 검증하는 미들웨어". 주니어가 코드를 훑어볼 때 빠르게 파악하는 용도입니다.

L2 — 기능 설명. 파라미터, 리턴값, 예외 상황, 사용 예시. 기존 **docstring**의 상위호환이지만, **LLM**이 실제 코드 로직을 분석해서 생성하므로 **docstring**이 없거나 **outdated**된 코드에도 동작합니다.

L3 — 아키텍처 맥락. 이 코드가 전체 시스템에서 어디에 위치하는지. "인증 미들웨어 체인의 두 번째 단계이며, JWT 디코딩 결과를 받아서 권한 체크 모듈로 넘기는 역할". 의존성 그래프 정보를 활용해서 자동 생성합니다.

L4 — 히스토리/의사결정 맥락. "원래 동기식이었으나 **PR #234**에서 비동기로 전환. 이유: 토큰 검증이 외부 **API** 호출을 포함하게 되면서 병목 발생". **git** 히스토리와 **PR** 디스커션에서 추출합니다.

Living Documentation

핵심은 이 문서가 코드와 함께 업데이트된다는 것입니다. 두 가지 방식으로 동작합니다.

능동적 업데이트: 사용자가 파일을 저장하면, 변경된 함수/클래스의 문서를 백그라운드에서 재생성합니다. 사용자가 요청하지 않아도 자동으로 최신 상태를 유지합니다.

변경 감지 알림: **git**에서 다른 사람이 코드를 바꿨는데 기존 문서와 충돌이 생기면, "이 코드가 변경되어 문서가 **outdated**되었습니다"라고 알림을 줍니다. **PR** 프로세스에 연동하면 "이 **PR**로 인해 다음 문서가 업데이트되어야 합니다"라는 자동 코멘트를 달 수도 있습니다.

문서의 저장 위치

문서를 레포에 마크다운 파일로 커밋하는 방식과, 별도 **DB**에 저장하는 방식 중 선택해야 합니다. 사내 도구이고 팀 공유가 목적이면, 별도 **DB**에 저장하되 **VS Code**에서 바로 보여주는 게 낫습니다. 레포를 문서로 어지럽히지 않으면서도 모든 팀원이 같은 문서를 보게 됩니다.

세 기능이 시너지를 내는 방식

이 세 기능은 독립적으로도 가치가 있지만, 합쳐지면 더 강력해집니다.

Q&A 챗봇이 답변할 때 의존성 그래프와 자동 생성 문서를 컨텍스트로 활용합니다. 단순히 코드를 읽어서 답하는 게 아니라, 구조적 관계와 히스토리 맥락까지 포함된 답변이 나옵니다.

임팩트 분석 결과를 볼 때 각 영향받는 노드의 자동 문서를 함께 보여줍니다. "이 함수를 바꾸면 **payment** 모듈의 **process_refund**에 영향갑니다"만이 아니라 "**process_refund**는 환불 처리의 핵심 로직이며, 최근 2주 내 3번 변경되었고, **bus factor**가 1입니다"까지 한 눈에 보입니다.

자동 문서화가 의존성 그래프를 참조해서 **L3**(아키텍처 맥락) 문서를 생성합니다. 코드만 보고는 생성할 수 없는 맥락이 그래프 정보 덕분에 가능해집니다.

로컬 서버 아키텍처를 택한 이유

VS Code Extension만으로 모든 걸 처리하지 않고 별도 로컬 백엔드 서버를 두는 이유가 몇 가지 있습니다.

무거운 작업 분리. 레포 전체를 파싱하고, 의존성 그래프를 구축하고, 임베딩을 생성하는 건 시간과 리소스가 많이 드는 작업입니다. **Extension** 프로세스 안에서 하면 에디터가 버벅거립니다.

상태 유지. 벡터DB, 의존성 그래프, 문서 저장소 같은 상태를 서버에 유지하면, **Extension**이 재시작되어도 데이터가 살아있습니다.

팀 공유 가능성. 초기에는 로컬에서 돌리더라도, 나중에 사내 서버에 올리면 팀원 전체가 같은 인덱스를 공유할 수 있습니다. 한 사람이 인덱싱하면 팀 전체가 혜택을 받는 구조가 됩니다.

기술 자유도. 백엔드를 Python으로 짜면 **tree-sitter**, **NetworkX**, **PyDriller** 같은 강력한 라이브러리를 자유롭게 쓸 수 있습니다. **Extension** 안에서는 이런 도구들을 쓰기 어렵습니다.

설치부터 사용까지 전체 흐름

1단계: 설치와 초기 세팅

개발자가 VS Code 마켓플레이스에서 이 **Extension**을 설치합니다. 설치하면 사이드바에 새 아이콘이 하나 생깁니다. 처음 클릭하면 "분석할 레포를 선택하세요"라는 화면이 뜹니다.

현재 워크스페이스에 열린 레포를 선택하면, 백그라운드에서 인덱싱이 시작됩니다. 이게 첫 번째이자 가장 중요한 단계인데, 레포 전체의 코드를 파싱하고, 함수/클래스 관계를 분석하고, git 히스토리를 읽어서 지식 데이터베이스를 구축하는 과정입니다.

인덱싱 동안 하단 상태바에 진행률이 표시됩니다. "Indexing: 파싱 중 (234/500 files)..." 같은 식으로요. 중간 규모 레포(파일 수백 개)면 몇 분, 대형 레포면 십 수 분 정도 걸립니다. 인덱싱이 끝나면 "Ready" 상태가 되고, 이때부터 모든 기능을 쓸 수 있습니다.

중요한 건, 인덱싱은 처음 한 번만 오래 걸린다는 겁니다. 이후에는 파일이 변경될 때 해당 파일만 증분 업데이트하므로 거의 실시간입니다.

2단계: 일상적인 사용 — 세 가지 진입점

인덱싱이 끝난 후, 개발자가 이 도구를 쓰는 방식은 크게 세 가지입니다.

진입점 **A**: 사이드바 채팅 — "레포에 대해 물어보기"

사이드바에 채팅 패널이 있습니다. 여기에 자연어로 질문합니다.

예를 들어 새로 합류한 개발자가 "이 레포에서 결제 처리는 어떻게 돌아가?"라고 입력합니다.

도구가 하는 일은 이렇습니다. 먼저 질문을 임베딩해서 벡터DB에서 관련 코드 청크들을 찾습니다. 결제 관련 함수, 클래스, 모듈이 검색됩니다. 그다음 검색된 코드들의 의존성 관계를 그래프에서 가져옵니다. 단순히 "이런 파일들이 있다"가 아니라 "결제 요청이 들어오면 A 모듈이 받아서 B로 넘기고, B가 외부 PG사 API를 호출하고, 결과를 C가 DB에 저장한다"는 흐름을 파악할 수 있습니다. 마지막으로 이 모든 정보를 LLM에 전달해서 사람이 읽을 수 있는 설명으로 만듭니다.

답변은 채팅 패널에 표시되는데, 단순 텍스트가 아니라 참조된 파일 링크가 함께 나옵니다. "payment/processor.py의 process_payment 함수"라고 나오면, 클릭하면 해당 파일의 해당 라인으로 바로 점프합니다.

여기서 핵심적인 차별점이 하나 있습니다. 도구가 지금 열려 있는 파일을 인식한다는 겁니다. payment/processor.py를 열어놓고 "이거 어떻게 돌아가?"라고만 물어도, "이거"가 현재 파일을 가리킨다는 걸 알고, 그 파일 중심으로 답변합니다. api/routes.py를 열어놓고 같은 질문을 하면 라우트 관점에서 답변이 달라집니다. 개발자의 현재 작업 맥락을 이해하는 거죠.

follow-up 질문도 됩니다. "그 process_payment 함수는 왜 비동기로 짰어?"라고 물으면, git 히스토리에서 해당 함수가 비동기로 바뀐 커밋을 찾아서 "PR #187에서 변경됨, 커밋 메시지에 따르면 PG사 응답 지연 문제 때문"이라고 맥락까지 설명합니다.

진입점 **B**: 코드 위에서 직접 — "이 코드 뭔데?"

채팅까지 갈 필요도 없이, 코드를 보다가 바로 정보를 얻는 방식입니다. 두 가지가 있습니다.

Hover(마우스 올리기). 함수나 클래스 이름 위에 마우스를 올리면, 기존 VS Code는 타입 정보 정도만 보여줍니다. 이 **Extension**이 설치되면 거기에 추가 정보가 붙습니다. 한 줄 요약("사용자 인증 토큰을 검증하는 미들웨어"), 이 함수를 호출하는 곳 수(12곳), 최근 변경(2주 전, 김개발), bus factor(⚠ 1명) 같은 핵심 메타데이터를 한눈에 볼 수 있습니다.

우클릭 컨텍스트 메뉴. 코드를 선택하고 우클릭하면 기존 메뉴에 새 옵션들이 추가됩니다.

"Explain This Code" — 선택한 코드 블록에 대해 채팅 패널에서 자동으로 설명을 시작합니다. 코드를 복사해서 채팅에 붙여넣고 "이거 설명해줘"라고 칠 필요가 없습니다.

"Analyze Impact" — 이 함수/클래스를 수정하면 어디가 영향받는지 분석합니다. 이게 진입점 C로 연결됩니다.

"Show History" — 이 코드 블록의 변경 이력을 시간순으로 보여줍니다. 단순 git log가 아니라 "무엇이 왜 바뀌었는지"를 요약해서 보여줍니다.

진입점 C: 임팩트 분석 패널 — "이거 바뀌도 되나?"

우클릭에서 **"Analyze Impact"**를 선택하거나, 채팅에서 "이 함수 바꾸면 어디 영향가?"라고 물으면, 에디터 옆에 임팩트 분석 패널이 열립니다.

이 패널에는 두 가지가 표시됩니다.

의존성 그래프 시각화. 선택한 함수를 중심에 놓고, 이 함수에 영향받는 코드들이 방사형으로 펼쳐지는 그래프입니다. 1차 의존(직접 호출하는 곳)은 가까이, 2차·3차 간접 의존은 멀리 배치됩니다. 각 노드의 색상이 위험도를 나타냅니다 — 빨간색은 직접 영향 + 테스트 없음(위험), 주황색은 간접 영향, 초록 테두리는 테스트가 커버하는 코드(안전). 노드를 클릭하면 해당 코드로 점프합니다.

영향 요약 리스트. 그래프 아래에 텍스트로 정리된 영향 목록이 나옵니다. "직접 영향 5곳, 간접 영향 12곳. 주의: `payment/refund.py`의 `process_refund`는 테스트가 없고 `bus factor`가 1입니다." 이런 식으로 어디를 특히 조심해야 하는지 하이라이트해줍니다.

이 화면에서 영향받는 파일 중 하나를 클릭하면 그 파일이 에디터에 열리고, 영향받는 라인이 하이라이트됩니다. 거기서 다시 **"Explain This Code"**를 하거나 추가 임팩트 분석을 체이닝할 수 있습니다.

3단계: 백그라운드에서 계속 돌아가는 것들

사용자가 명시적으로 호출하지 않아도 백그라운드에서 계속 동작하는 기능들이 있습니다.

증분 인덱싱. 파일을 저장할 때마다, 변경된 파일의 코드를 다시 파싱하고 임베딩을 업데이트합니다. 의존성 그래프도 해당 부분만 갱신됩니다. 사용자가 항상 최신 상태의 지식 베이스와 대화하는 셈입니다.

문서 자동 업데이트. 코드가 바뀌면 해당 코드의 자동 생성 문서(L1 한 줄 요약, L2 기능 설명)도 백그라운드에서 재생성됩니다. 다음에 그 코드 위에 마우스를 올리면 이미 업데이트된 설명이 보입니다.

Stale 문서 감지. `git pull` 등으로 다른 사람의 변경사항이 들어오면, 기존 문서와 현재 코드 사이에 불일치가 있는지 체크합니다. 불일치가 감지되면 해당 파일에 작은 경고 마커를 표시합니다. "이 파일의 문서가 최신 코드와 맞지 않을 수 있습니다."

4단계: 팀 레벨 사용 (확장 시나리오)

초기에는 각 개발자의 로컬 머신에서 돌아가지만, 백엔드 서버를 사내 서버로 올리면 팀 전체가 공유하는 형태가 됩니다.

이때 추가로 가능해지는 것들이 있습니다. 한 사람이 질문한 로그가 쌓이면서 "자주 묻는 질문" 패턴이 보이기 시작합니다. "신규 입사자 3명이 모두 결제 모듈에 대해 비슷한 질문을 했다" → 이 패턴을 감지해서 자동 **FAQ**를 생성하거나, 해당 모듈에 더 상세한 문서가 필요하다는 신호로 활용할 수 있습니다.

팀별 **Ownership** 대시보드도 가능해집니다. "이 모듈은 사실상 한 사람만 이해하고 있다"는 **Bus factor** 경고를 팀 리드가 볼 수 있게 되면, 지식 편중 문제를 조기에 발견하고 대응할 수 있습니다.

한 줄 요약

개발자가 **VS Code**에서 코드를 보다가 궁금하면 채팅으로 물어보고, 함수 위에 마우스 올려서 핵심 정보 확인하고, 수정 전에 우클릭으로 영향 범위를 파악하는 — 이 세 가지가 끊임 없이 연결되는 도구입니다. 그리고 이 모든 게 문서화 레이어 위에서 돌아가서, 코드가 바뀌어도 지식이 항상 최신 상태를 유지합니다.

파싱 자체에는 **AI**가 필요 없다

코드를 파싱해서 "이 파일에 함수가 몇 개 있고, 뭘 **import** 하고, 뭘 호출하는지" 추출하는 건 **tree-sitter** 같은 전통적인 파서로 충분합니다. 이건 프로그래밍 언어의 문법 규칙에 따라 기계적으로 처리하는 거라 **AI**가 개입할 이유가 없고, 오히려 **AI**를 쓰면 부정확해집니다. 함수 시그니처나 **import** 관계 같은 건 **100%** 정확해야 하는데, **LLM**은 이런 걸 가끔 빠뜨리거나 환각을 만들거든요.

의존성 그래프 구축도 마찬가지입니다. **A** 함수가 **B** 함수를 호출한다는 관계는 **AST**에서 기계적으로 추출하는 거지 **AI**가 판단할 문제가 아닙니다.

git 히스토리 분석도 **AI** 없이 됩니다. 커밋 로그 파싱, **blame** 정보 추출, 기여도 계산은 전부 룰 기반입니다.

AI가 필요한 곳은 명확하다

AI가 필요한 건 사람이 읽을 수 있는 "이해"를 생성하는 단계입니다. 구체적으로 네 군데입니다.

첫째, 임베딩 생성. 코드 청크를 벡터로 변환해서 "의미적으로 비슷한 코드"를 검색할 수 있게 하는 거. "결제 처리"라고 질문하면 `process_payment` 함수가 검색되려면 의미적 유사도를 계산할 수 있어야 합니다.

둘째, **Q&A** 답변 생성. 검색된 코드 + 의존성 정보 + `git` 히스토리를 종합해서 자연어 답변을 만드는 거. 이건 LLM 없이는 불가능합니다.

셋째, 자동 문서 생성. 코드를 보고 "한 줄 요약", "기능 설명", "아키텍처 맥락"을 사람이 읽을 수 있는 글로 쓰는 거.

넷째, 히스토리 요약. 커밋 메시지 10개를 보고 "이 함수는 원래 동기식이었는데 성능 문제로 비동기로 전환됐다"라고 요약하는 거.

로컬 AI vs 클라우드 API — 현실적 판단

여기서 핵심 질문은 "로컬에서 돌릴 수 있는 모델로 이 네 가지가 충분히 되느냐"입니다.

임베딩: 로컬로 충분하다

임베딩은 로컬 모델로 해도 품질이 꽤 좋습니다. 코드 특화 임베딩 모델 중에 로컬에서 돌릴 수 있는 게 여럿 있고, 모델 크기도 작아서 GPU 없이 CPU만으로도 충분한 속도가 나옵니다. 인덱싱 시점에 한 번 돌리고 이후에는 변경분만 처리하니까 속도가 크게 중요하지도 않습니다.

Q&A와 문서 생성: 여기가 갈림길이다

이건 모델 품질에 직결되는 부분이라 선택이 필요합니다.

로컬 모델을 쓰는 경우, **Ollama** 같은 도구로 로컬에서 LLM을 돌릴 수 있습니다. 장점은 코드가 외부로 나가지 않아서 보안이 완벽하고, **API** 비용이 없다는 것. 단점은 품질입니다. 현재 로컬에서 돌릴 만한 모델(7B-13B 파라미터급)은 코드 이해와 긴 맥락 처리에서 클라우드 모델 대비 확실히 떨어집니다. 특히 여러 파일에 걸친 아키텍처 맥락을 설명하거나, 복잡한 의존성 체인을 요약하는 데서 한계가 보입니다. 또한 로컬 모델을 쓸 만한 성능으로 돌리려면 개발자 머신에 어느 정도 GPU가 있어야 합니다. 사내 도구라면 팀원들 머신 스펙이 다 다를 수 있어서 이게 제약이 됩니다.

클라우드 **API**를 쓰는 경우, **Claude API**나 **OpenAI API**를 쓰면 품질은 확실히 좋습니다. 긴 컨텍스트 처리, 코드 이해, 다국어 코드 설명 전부 로컬 모델보다 훨씬 낫습니다. 단점은 코드가 외부 서버로 나간다는 것과 **API** 비용입니다. 사내 도구에서 소스코드를 외부 API로 보내는 게 보안 정책상 허용되는지가 첫 번째 관문이고, 사용량에 따라 비용이 쌓이는 것도 고려해야 합니다.

추천하는 현실적 구조

두 가지를 섞는 하이브리드 구조가 가장 현실적입니다.

파싱, 의존성 분석, **git** 분석은 **AI** 없이 로컬에서 처리합니다. 이걸 선택의 여지가 없고 로컬이 맞습니다.

임베딩은 로컬 모델로 돌립니다. 충분한 품질이 나오고 비용도 없습니다.

Q&A 답변과 문서 생성은 설정으로 선택할 수 있게 만듭니다. **Extension** 설정에서 "**LLM Provider**"를 고르게 하는 거죠 — **Claude API**, **OpenAI API**, 또는 **Ollama**(로컬). 보안이 민감한 회사는 로컬을 쓰고, 품질을 원하는 팀은 클라우드 **API**를 쓰고, 회사가 자체 **LLM** 서버를 운영하면 그걸 연결하면 됩니다.

이 구조의 핵심은, **AI** 레이어를 교체 가능하게 추상화하는 겁니다. 코드 분석 엔진과 **AI** 레이어 사이에 명확한 인터페이스를 두면, **LLM**이 뭐가 되든 나머지 시스템은 그대로 돌아갑니다. 나중에 더 좋은 로컬 모델이 나오면 갈아끼우기만 하면 되고요.

비용 현실 감각

클라우드 **API**를 쓴다면 비용이 어느 정도인지 감을 잡아두는 게 좋습니다.

초기 인덱싱 (문서 생성). 파일 **500**개짜리 레포에서 각 함수/클래스에 대해 문서를 생성한다고 하면, 엔티티가 대략 **2000-3000**개. 각각에 대해 짧은 **LLM** 호출을 한다고 하면 총 입출력 토큰이 수백만 토큰 수준. **Claude Sonnet** 기준으로 몇 달러 정도입니다. 한 번만 하면 되니까 부담이 크지 않습니다.

일상 사용 (**Q&A**). 개발자 한 명이 하루에 질문을 **10-20**번 한다고 하면, 각 질문에 코드 컨텍스트를 포함해서 수천 토큰. 하루에 개발자당 몇십 센트 수준. 팀 **10**명이면 한 달에 수십 달러 정도.

증분 업데이트. 파일 저장 시마다 문서를 재생성하면 호출이 많아질 수 있어서, 이걸 배치로 모아서 처리하거나, 일정 시간 간격으로 업데이트하는 게 비용 효율적입니다.

요약하면

파싱과 구조 분석에는 **AI**가 아예 필요 없고, "사람이 읽는 출력"을 만드는 네 군데에만 **AI**가 필요합니다. 임베딩은 로컬로 충분하고, 나머지는 품질과 보안 사이에서 선택입니다. 가장 현실적인 답은 **LLM provider**를 교체 가능하게 만들어서 상황에 맞게 쓸 수 있게 하는 겁니다.

Brainstorming

1. Repo Onboarding Buddy

- a. 신입 직원들의 예상 FAQ 를 모아 시니어가 답변을 하게 하고, 이후로는 시니어 답변을 가지고 AI training
- b. 채팅 방식으로 신입의 질문이 시니어에게 전달되어 답변을 받고, 이 답변은 이후 유사 질문에 대한 답으로 사용
- c. AI 까지 입히지 않아도 될수도, 모범답안 DB 활용하는 방식
- d. Comments around code 가 아니라 채팅 방식으로 예상 질문에 대한 답을 남기는 방식
- e. 시니어의 답변 활동이 로그에 남아 인사고과에 활용할 수 있는 평가 지표로 사용할 수도 있음
- f. 개발 스트럭처를 AI 활용해서 잡아두고 이를 활용해서 ...
 - i. AI 활용 최소화: 예상 질문과 답변 AI 로 만들어 저장하고 사용하는 방식
 - ii. AI 적극 활용: 개발 스트럭처에 기반하여 질문에 답변하도록 AI 활용
=> 둘 중 어느 방식이든 시니어의 컨펌이나 수정이 있는 방향이 좋을 것

같은

- g. 클러드를 사용하지 않고 다음 질문에 답할 수 있는가?
 - i. “이 코드를 수정하면 다른 어느 곳에 문제가 생겨?”
 - ii. “이 변수/함수는 뭘 위한 거야?”
 - iii. “이 부분과 연결되어 이해해야 하는 다른 부분은 어디야?”
- h. 어느 부분에서 혼다 개발자들이 불편함을 느끼고 있어서 이러한 솔루션을 필요로 하는지 확인 필요
- i. 혼다에서 코드베이스에 AI 접근을 어디까지 허용할 것인지 확인 필요
 - i. 오피시던 쓰면 안되나...?

2. Auto Documentation

- a. Naive approach: 코드 수정 후 커밋할 때 수정 내용을 다큐먼트에 추가할 수 있는 윈도우나 터미널 메시지 띄우기
- b. AI approach: 클러드랑 다르게 뭐지...?

3. Usage Tracking Layer

- a. Git history parsing -> Data extraction -> Analysis -> Classification

종합: 코멘트/주석 남기고 슬랙/기타메신저 통해 이전 질답 기록 찾는 것을 넘어 코드에 대한 내용을 VS Code 에서 확인할 수 있도록 (구글닥 콜라보할 때 처럼 VS Code 를 사용할 수 있으면 좋겠다)

경쟁 모델: Live Share, CodeTogether

Things to Discuss

1. Executive Summary

- **현황:** "Repo Onboarding Buddy"는 신입 개발자의 온보딩 비용 절감과 시니어 리소스 확보를 목표로 함.
- **문제점:** 기존 AI 코딩 툴(Copilot 등)과의 차별성 부족, 시니어의 답변 부담(Cold Start), 데이터 최신성 유지(Data Rot)의 어려움이 식별됨.
- **제언:** "단순 질답 봇"에서 ***사내 맥락(Context) & 히스토리 저장소***로 개념을 확장하고, 시니어의 답변 활동을 강제하기보다 자연스러운 워크플로우로 통합하는 피벗(Pivot)이 필요함.

2. 주요 쟁점 분석 (Key Critical Points)

(1) Value Proposition (가치 제안)

- **Current Idea:** 코드의 기능을 설명해주는 챗봇.
- **Critique:** Claude, Copilot 등 상용 LLM이 이미 코드를 완벽하게 해석함. 단순히 "이 함수가 뭐냐"를 답하는 것은 경쟁력이 없음.
- **New Direction:** ***"왜 이렇게 짰는가?"(Tribal Knowledge)**를 저장해야 함. 코드가 작성된 배경, 비즈니스적 요구사항, 건드리면 안 되는 레거시 로직 등 '사람만 아는 정보'를 캡처하는 데 집중해야 함.

(2) Data Strategy & Cold Start (데이터 전략)

- **Current Idea:** 시니어가 신입 질문에 답하면 이를 학습(Training) 데이터로 활용.
- **Critique:** 초기 데이터가 전무한 상태에서 시니어에게 답변 노동을 강요하게 됨. 또한, 답변 데이터는 코드가 수정되는 순간 '오답'이 될 위험이 큼.
- **New Direction:** AI 학습(Training) 대신 **RAG**(검색 증강 생성) 방식을 채택. 코드 수정 시 관련 질답을 '만료(Expired)' 처리하는 데이터 검증 로직 필수.

(3) Incentive & KPI (동기 부여)

- **Current Idea:** 답변 활동을 인사고과에 반영.
- **Critique:** 질보다 양을 채우기 위한 무의미한 답변(Gaming)을 유발하거나, 개발자 문화에 거부감을 줄 수 있음.
- **New Direction:** 답변의 '작성 횟수'가 아닌, 해당 지식이 추후 얼마나 '재사용(Viewed/Helpful)' 되었는지를 측정하여 기여도를 평가해야 함.

3. 기술적 구현 및 로드맵 제안

- 플랫폼: VS Code Extension (채팅형보다는 코드 라인별 스레드 방식 권장).
- 핵심 기능 전환:
 - AS-IS: Q&A Chatbot (실시간 대화 중심)
 - TO-BE: **Context Layering Tool** (코드 위에 설명 레이어를 씌우는 비동기 지식 저장소)

4. Action Items (팀 논의 사항)

- 혼다 측에 "코드 설명" vs "히스토리 관리" 중 무엇이 더 큰 **Pain Point** 인지 재확인.
- AI 모델 학습(**Training**)을 포기하고 **RAG** 아키텍처로 선회할 것인지 결정.
- 시니어의 답변 부담을 줄일 수 있는 자동화 방안(**Slack** 연동 등) 아이디어이션.

자기소개

윤하:

혼다에서 제시한 프로젝트 소개 (Code Repo Knowledge Layer 탭 확인)

재윤 **Pain point/Transition:**

우리가 이해한 내용 소개 및 우리가 이해한 게 맞는지 확인 (우리 동아리에서 생각하는 페인포인트 문제점을 먼저 설명하고 회사에서 느끼는 점과 같은지를 확인해야 할 것 같다)
이메일 보낸 질문들 확인 ()

관호:

1. Nature of Onboarding Bottlenecks: When new hires face difficulties, is their primary challenge understanding the technical logic (e.g., "What does this function do?") or understanding the historical/business context (e.g., "Why was this implemented this way?" or "What legacy issues should I be aware of?")? If the latter is the major pain point, we aim to focus our solution on capturing "tribal knowledge" rather than generic code explanation.

- 2 functional standpoint:
 - Q&A: an accurate answer to the question
 - Design a system that could automatically generated/dynamically live **document** of answers to the questions
 - open source project - not a person-specific - not relying on one person's perspective or idea

종훈:

2. Current Communication Channels: How do senior developers currently answer questions from new hires? Do these interactions mostly happen via instant messaging (Slack/Teams), issue trackers (Jira), or verbal communication? We are exploring ways to leverage existing communication logs to minimize the manual documentation burden on your senior developers.

- 신입들이 어떤 채널로 답변을 받았을 때 만족했는지 또는 어떤 채널을 선호하는지에 대한 가늠을 하고 있는지
- 반대로 시니어들의 설명이 어떤 채널로 전달되었을 때 가장 효과적인지 (신입 입장, 시니어 입장 둘 다 들어보기로)
- TLDR: Too Long; Didn't Read - make it as short and clear as possible
- Unstructured, semi-structured data into **cross-connected networks** - knowledge representation
- To understand the whole, need to understand the part. But to understand the part, need to understand the whole.
- Want to have a **doc** explaining what this repo is supposed to do

찬영:

3. AI Security & Policy: Could you clarify the internal policies regarding AI tools on your codebase? Specifically, are we permitted to use external LLM APIs (e.g., OpenAI, Anthropic), or does the solution require a strictly local/on-premise model due to security concerns?

- Honda has an arrangement with **OpenAI**.

- For this project: design the shell, then add their api
- Open source route

승연:

4. Development Environment: We are currently envisioning a solution integrated directly into VS Code as a plugin. Is this the primary development environment for your team, or should we consider compatibility with other IDEs?

- VSCode sounds a good starting point, expand to other tools/IDEs, build it as a open source

Fri: 9:00-10:00 PST, 1:00 - 2:00 PST

Tue, Thr afternoon

Ryan shared: <https://web4.ai/>

2월 23일(월)GM

Phase1: Due 3/9

1. Repo Onboarding Buddy -> 찬영, 승연, 재운
3. Usage Tracking Layer -> 종훈, 관호, 윤하

Phase2: Due 3/23

2. Auto Documentation / 커밋 전 후로 document 업데이트 될 수 있도록 플로우 연결

Stretch Goal

4. Interactive Code Map
5. Cross-team Impact Alerts

- 1) 필요한 오픈소스 찾아보기 ~2/24
-> 2/24 오후 9시에 줌 미팅
- 2) CoLab 대신 쓸 코드작성 프로그램?

Usage Tracking Layer

1. 기준 정하기 (뭐가 unused code or frequently used code?) / Set git history as the data source
2. 데이터 추출 -> Git history 등에서
3. 분류 방법 정하기 -> AST parsing / 시간 순으로도 분류 가능하고 import 되는 횟수로도 분류 및 중요도 판정 가능
4. 분류한 데이터들끼리 점수 체계를 만들기
5. Tableau -> visualization / 트리맵을 만들어서 색깔대로 중요도 표현

그리고 프로젝트 전체적인 틀을 보자면 한조가 전에 저희 구글닥에 올려놓은 것 같은데 읽어보면 전체적인 감이 오는 것 같아서, 구글 닥에서 프로젝트 AI 요약본 부분 안 읽어보신 분 있으면 읽어보시면 좋을 것 같습니다!

제가 생각해본 전체적인 틀을 말씀드리자면

Phase 1 때 진행할 것은

Repo Onboarding Buddy에서 Extension을 구현할 수 있도록 UI를 만들면 될 것 같고 (시간이 되면 시니어가 Q&A 방식으로 답변할 수 있는 창도 만들면 좋을 듯합니다.)

Usage Tracking Layer에서 말 그대로 코드 간의 의존도를 파악할 수 있는 그래프 / Layer를 만들면 될 것 같습니다.

그리고 Phase 2 때 진행할 것은

Auto Documentation이라고 써있기는 하지만 이것 포함해서

챗봇의 backend 적인 부분을 Phase 2 때 구현하면 어떨까 싶습니다. 챗봇에서 AI도 사용하지만 거기에 Usage Tracking Layer와 Auto Documentation 또한 참고해서 답변하도록 구현하면 더 정확한 답변 제공이 가능할 것 같습니다.

그래서 **Phase 2** 부분이 실질적으로 해야할 부분들이 많은 것 같아 **Phase 1**은 최대한 빨리 **Due** 안에 끝내고 **Phase 2**로 넘어가 이 부분에 집중하는 것이 좋을 것 같아요~!

윤하

PyDriller(=Git 분석): “누가/언제/얼마나 이 코드를 건드렸나?”(사람·시간·변경)

CodeQL(=정적 코드 분석): “코드가 코드 안에서 어떻게 연결/참조되고 있나?”(의존·참조·호출)

찬영

참고할 만한 오픈소스 코드

Repo Sense -> Git 레포에서 누가 언제 어디에 얼마나 기여했는지”를 뽑아서, 웹 리포트(대시보드)로 시각화해주는 툴 / 웹사이트에 알려주는 툴

-> Git 분석에 강함

<https://github.com/reposense/RepoSense>

Sourcegraph -> 조직의 코드베이스를 중앙에서 인덱싱해, 빠른 코드 검색과 구조 탐색(정의/참조 추적) 및 AI 질의를 제공하는 코드 인텔리전스 플랫폼

-> 여기는 근데 웹 플랫폼이 메인이라 우리는 **IDE extension**에 메인으로 치중? / IDE 확장이 있다고 하기는 함

-> 오픈 소스가 아니라고 함.

샘플 코드

Pallets/flask -> Python으로 웹 서버와 웹 애플리케이션을 쉽게 만들 수 있게 해주는 가벼운 웹 프레임워크 (Python 구조)

<https://github.com/pallets/flask>

Pallets/itsdangerous -> 데이터에 암호학적 서명을 붙여서 위조되지 않았는지 검증할 수 있게 해주는 작은 Python 보안 라이브러리

<https://github.com/pallets/itsdangerous>

Whisper X -> 음성 파일을 텍스트로 변환하고, 단어별 타임스탬프와 화자 구분까지 해주는 고급 음성 인식 도구 / ML 활용한 툴

<https://github.com/m-bain/whisperX>

<https://github.com/tree-sitter/tree-sitter>

<https://github.com/networkx/networkx> - Python 기반의 그래프 이론 라이브러리
노드와 엣지로 구성된 네트워크를 생성하고, 중심성(centrality), PageRank, 연결성, 최단 경로 등의 다양한 그래프 알고리즘을 계산

Phase 1(Q&A): Tree-sitter + Sentence-Transformers + FAISS + LlamaIndex + FastAPI + (Ollama 옵션)

Phase 2(Living Docs): MkDocs(Material) + GitHub Webhooks/PR Comments API

<https://www.mkdocs.org/>

<https://docs.github.com/en/webhooks/about-webhooks>

Phase 3(Usage Tracking): CodeQL + PyDriller + (Vulture 옵션) + Streamlit

Phase 4(Code Map/Impact): Cytoscape.js + VS Code Webview + (OpenTelemetry 옵션)

2/24(화) 미팅

02/26/26 Updated

1. 지금 만든 것

CLI: repoknow

API 서버: repoknow-api (FastAPI)

핵심 코드 위치:

cli.py

api.py

codeql_scan.py

git_metrics.py

score.py

사용 문서:

[README.md](#)

2. 어떻게 사용하나 (실행 순서)

설치

cd /Users/yhkim/code-compass/usage-tracking-layer

source .venv/bin/activate

pip install -e . --no-build-isolation

분석 실행 (CLI)

repoknow scan --repo /path/to/target-git-repo

API로 쓰려면 서버 실행

repoknow-api

브라우저에서

http://127.0.0.1:8000/ (자동 docs 이동)

<http://127.0.0.1:8000/docs>

3. 결과물 어디 생기나

대상 레포 내부: **/path/to/target-git-repo/.repoknow/data/**

파일:

imports.csv

modules.csv

internal_edges.csv

file_git_metrics.csv

file_scored.csv

4. 결과 해석법

imports.csv: 누가 어떤 모듈을 import했는지 원본 이벤트

modules.csv: 파일 ↔ 모듈명 매핑

internal_edges.csv: 내부 의존성 그래프 (src_file -> dst_file)

file_git_metrics.csv: 변경 이력/오너십

authors_180d, top1_share로 bus factor 위험 판단

file_scored.csv: 최종 통합 결과

ref_count: 내부에서 이 파일을 참조하는 정도 (영향도)

category: Core, Stable, DeadCandidate, Other, NonCode

5. 해석 규칙

DeadCandidate는 “삭제 확정”이 아니라 검토 후보

impact 결과는 현재 1-hop 직접 영향 기준

Python 동적 특성 때문에 정적 분석은 오탐/누락 가능성이 있어 런타임 신호(추후 OTel)로 보강하는 게 맞음

6. 실무에서 바로 보는 API

POST /scan: 분석 실행

GET /summary: 카테고리 요약

GET /top-imports: 많이 쓰는 모듈

GET /bus-factor-alerts: 지식 편중 위험

GET /impact?file=...: 변경 영향 파일

GET /file-info?file=...: 특정 파일 상세

02/25/26 Updated

GitHub – Usage Tracking Layer Guide (repoknow 실행)

1. 저장소 클론 후 이동
git clone <repo-url>
cd usage-tracking-layer
2. 가상환경 + 설치
python3 -m venv .venv
source .venv/bin/activate
pip install -e . --no-build-isolation
3. Python 프로젝트 분석 (추천)
repoknow scan --repo /path/to/python-git-repo
4. 결과 확인 위치
/path/to/python-git-repo/.repoknow/data/

생성되는 주요 파일:

- imports.csv (CodeQL import 이벤트)
- modules.csv (모듈 맵)
- internal_edges.csv (내부 의존성 그래프)
- file_git_metrics.csv (Git churn/ownership/bus factor 지표)
- file_scored.csv (통합 점수/분류)

5. 영향도(임팩트맵)질의 예시

repoknow impact --repo /path/to/python-git-repo --file src/some_module.py

비파이썬 프로젝트면

repoknow scan-git --repo /path/to/git-repo

자주 막히는 포인트

- repo는 반드시 .git 있는 프로젝트 루트 폴더
- scan-codeql/scan은 Python 파일(.py) 있는 레포에서만 사용
- CodeQL CLI 없으면 scan-codeql은 동작 안 함 (scan-git는 가능)

한 줄 설명

- CodeQL로 코드 구조/의존성(누가 누구를 import하는지)
- PyDriller로 변경 이력/오너십(누가 얼마나 자주 바꾸는지)
- 둘을 합쳐서 온보딩/임팩트맵/자동문서화의 베이스 데이터를 만든다

확인 방법

1. 먼저 실행

```
export TARGET_REPO="/Users/yhkim/내프로젝트레포경로"  
repoknow scan --repo "$TARGET_REPO"
```

2. 결과물 생성 확인

```
ls -lh "$TARGET_REPO/.repoknow/data"
```

여기서 보이면 정상:

```
- imports.csv  
- modules.csv  
- internal_edges.csv  
- file_git_metrics.csv  
- file_scored.csv
```

3. 결과물 내용 빠르게 보기 (추천)

```
python - <<'PY'  
import os, pandas as pd  
base = os.path.join(os.environ["TARGET_REPO"], ".repoknow", "data")  
for f in  
["imports.csv", "modules.csv", "internal_edges.csv", "file_git_metrics.csv", "file_scored.csv"  
"]:  
    p = os.path.join(base, f)  
    df = pd.read_csv(p)  
    print("\n==", f, "==" )  
    print("shape:", df.shape)  
    print("columns:", list(df.columns))  
    print(df.head(5).to_string(index=False))  
PY
```

4. “어떻게 결과 나왔는지” 핵심 확인 포인트

• Top imports (자주 import되는 모듈)

```
python - <<'PY'  
import os, pandas as pd  
p = os.path.join(os.environ["TARGET_REPO"], ".repoknow/data/imports.csv")  
df = pd.read_csv(p)  
print(df["imported_module"].value_counts().head(20).to_string())  
PY
```

• Bus factor 후보 (한 사람에게 지식 쏠림)

```
python - <<'PY'  
import os, pandas as pd  
p = os.path.join(os.environ["TARGET_REPO"], ".repoknow/data/file_git_metrics.csv")  
df = pd.read_csv(p)  
alerts = df[(df["authors_180d"] <= 2) & (df["top1_share"].fillna(0) >= 0.75)]  
print(alerts[["file", "authors_180d", "top1_author", "top1_share", "churn_lines_180d"]].head(  
20).to_string(index=False))  
PY
```

• 분류 결과 (Core/Stable/DeadCandidate/NonCode)

```
python - <<'PY'  
import os, pandas as pd  
p = os.path.join(os.environ["TARGET_REPO"], ".repoknow/data/file_scored.csv")
```

```
df = pd.read_csv(p)
print(df["category"].value_counts(dropna=False).to_string())
print("\nTop 영향 후보(ref_count 높은 순):")
print(df.sort_values(["ref_count", "churn_lines_180d"],
ascending=False)[["file", "ref_count", "category"]].head(20).to_string(index=False))
PY
```

5. 내 파일(특정 파일) 기준으로 확인하는 법

- 그 파일 점수/분류 보기

```
python - <<'PY'
import os, pandas as pd
target = "src/your_file.py" # 바꿔서 사용
p = os.path.join(os.environ["TARGET_REPO"], ".repoknow/data/file_scored.csv")
df = pd.read_csv(p)
print(df[df["file"] == target].to_string(index=False))
PY
```

- 그 파일 바꾸면 누가 영향 받는지 보기 (임팩트맵 1-hop)

```
repoknow impact --repo "$TARGET_REPO" --file "src/your_file.py"
```

6. 결과가 안 나오거나 이상하면 먼저 체크

```
ls -a "$TARGET_REPO" | head # .git 있는지
```

```
find "$TARGET_REPO" -type f -name "*.py" | head # Python 파일 있는지
```

02/24/26

최종 결과는 file_scored.csv

나머지는 그걸 만들기 위한 중간 테이블입니다.

흐름:

file_git_metrics.csv (Git 기반 변경/소유 지표)

imports.csv + modules.csv (CodeQL 추출 원천)

internal_edges.csv (내부 의존 엣지)

file_scored.csv (최종 조인/분류)

1) file_git_metrics.csv // PyDriller

파일 단위 Git 지표입니다. "최근 N일(지금은 180일)" 동안의 변경/저자/churn 정보.

file_git_metrics

file	commit_count_180d	authors_180d	churn_lines_180d	last_modified	top1_author	top1_share
hw03/src/ArrayExercises.java	1	1	14	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/src/ListExercises.java	1	1	33	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/src/MapExercises.java	1	1	27	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/tests/ArrayExercisesTest.java	1	1	55	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/tests/DessertTest.java	1	1	242	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/tests/ListExercisesTest.java	1	1	65	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw03/tests/MapExercisesTest.java	1	1	63	2025-08-29T07:36:43+09:00	dwwwang@berkeley.edu	1.0
hw04/src/IntList.java	1	1	119	2025-08-31T22:12:44-07:00	75148537+AngryAny@users.noreply.github.com	1.0
hw04/src/IntListMystery.java	1	1	41	2025-08-31T22:12:44-07:00	75148537+AngryAny@users.noreply.github.com	1.0
hw04/tests/IntListOptionalTests.java	1	1	15	2025-08-31T22:12:44-07:00	75148537+AngryAny@users.noreply.github.com	1.0
hw04/tests/IntListRequiredTests.java	1	1	266	2025-08-31T22:12:44-07:00	75148537+AngryAny@users.noreply.github.com	1.0
hw01/src/Arithmetic.java	1	1	2	2025-09-02T23:24:00-07:00	yhkimslv@berkeley.edu	1.0
hw02/src/DoubleUp.java	1	1	11	2025-09-03T00:27:51-07:00	yhkimslv@berkeley.edu	1.0
hw02/src/PrintIndexed.java	1	1	9	2025-09-03T00:27:51-07:00	yhkimslv@berkeley.edu	1.0
hw02/src/StarTriangle5.java	1	1	9	2025-09-03T00:27:51-07:00	yhkimslv@berkeley.edu	1.0
hw02/src/StarTriangleN.java	1	1	9	2025-09-03T00:27:51-07:00	yhkimslv@berkeley.edu	1.0

주요 컬럼

file: repo 상대 경로

commit_count_180d: 최근 180일 동안 이 파일을 건드린 커밋 수

authors_180d: 최근 180일 동안 이 파일을 수정한 작성자 수

churn_lines_180d: 추가+삭제 라인 합계 (변경량)

last_modified: 최근 수정 시각

top1_author: 가장 많이 기여한 작성자(변경량 기준)

top1_share: 해당 작성자의 churn 점유율 (0~1)

해석 팁

commit_count, churn_lines가 크면 "최근 활발히 바뀌는 파일"

authors_180d가 크면 "여러 명이 건드는 파일" (조율 필요 가능성)

top1_share가 1.0에 가까우면 특정 1인 의존 가능성

2) imports.csv

CodeQL가 추출한 "import 사실" 원천 데이터입니다. 아직 내부/외부 구분 전입니다.

imports

src_file	line	imported_module
Desktop/2026-Spring/cs188/tutorial/grading.py	17	__future__
Desktop/2026-Spring/cs188/tutorial/grading.py	17	__future__.print_function
Desktop/2026-Spring/cs188/tutorial/grading.py	18	html
Desktop/2026-Spring/cs188/tutorial/grading.py	19	time
Desktop/2026-Spring/cs188/tutorial/grading.py	20	sys
Desktop/2026-Spring/cs188/tutorial/grading.py	21	json
Desktop/2026-Spring/cs188/tutorial/grading.py	22	traceback
Desktop/2026-Spring/cs188/tutorial/grading.py	23	pdb
Desktop/2026-Spring/cs188/tutorial/grading.py	24	collections
Desktop/2026-Spring/cs188/tutorial/grading.py	24	collections.defaultdict
Desktop/2026-Spring/cs188/tutorial/grading.py	25	util
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	15	util
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	16	functools
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	16	functools.reduce
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	258	io
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	258	operator
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	300	math
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	311	util
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	311	gridworld
Desktop/2026-Spring/cs188/cs188-proj/tracking/busters.py	23	game
Desktop/2026-Spring/cs188/cs188-proj/tracking/busters.py	23	game.GameStateData

주요 컬럼

src_file: import를 하는 파일

line: import가 나온 줄 번호

imported_module: import된 모듈명

예시 해석

grading.py가 html, __future__ 등을 import했다는 기록

외부 라이브러리 import도 포함됨

3) modules.csv

레포(또는 분석 범위) 안에서 "파일이 어떤 모듈명으로 해석되는지" 매핑 테이블입니다.

file	module_name
Desktop/2026-Spring/cs188/tutorial/grading.py	grading
Desktop/2026-Spring/cs188/cs188-proj/reinforcement/textGridworldDisplay.py	textGridworldDisplay
Desktop/2026-Spring/cs188/cs188-proj/search/projectParams.py	projectParams
Desktop/2026-Spring/cs188/cs188-proj/tracking/busters.py	busters
Desktop/2026-Spring/cs188/cs188-proj/tracking/game.py	game
Desktop/2026-Spring/cs188/cs188-proj/tracking/projectParams.py	projectParams
Desktop/2026-Spring/cs188/cs188-proj/machinelearning/losses.py	losses
Library/Mobile Documents/.Trash/cs188 proj 2/multiagent/util.py	util
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/posit/positron/_vendor/pygments/lexers/pony.py	positron__vendor.pygments.lexers.pony
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/posit/positron/_vendor/pygments/lexers/dotnet.py	positron__vendor.pygments.lexers.dotnet
Desktop/2026-Spring/cs188/cs188-proj/tracking/bustersAgents.py	bustersAgents
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/posit/positron/_vendor/typing_extensions.py	positron__vendor.typing_extensions
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/Python/core/history.py	IPython.core.history
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/arm64/cp38/zmq/auth/base.py	zmq.auth.base
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/posit/positron/_vendor/cattr/strategies/_class_methods.py	positron__vendor.cattr.strategies._class_methods
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/posit/positron/_vendor/cattr/strategies/_subclasses.py	positron__vendor.cattr.strategies._subclasses
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/Python/utlis/wildcard.py	IPython.utlis.wildcard
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/jedi/plugins/stdlib.py	jedi.plugins.stdlib
.vscode/extensions/ms-python.python-2025.18.0-darwin-arm64/python_files/lib/jedisp/importlib_metadata/_adapters.py	importlib_metadata.adapters
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/parsio/python/token.py	parsio.python.token
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/debugpy/_vendored/pydevd/pydevd_attach_to_process/winappdbg.process	debugpy__vendored.pydevd.pydevd_attach_to_process.winappdbg.process
Library/Caches/co.posit.python.ShipIt/update.hWYcMRo/Positron.app/Contents/Resources/app/extensions/positron-python/python_files/lib/pykernel/py3/debugpy/_vendored/pydevd/pydevd_attach_to_process/debugpy__vendored.pydevd.pydevd_attach_to_process.winappdbg.process	debugpy__vendored.pydevd.pydevd_attach_to_process.winappdbg.process

주요 컬럼

file: 파일 경로

module_name: 그 파일이 제공하는 모듈명
해석 팁

imports.csv.imported_module와 modules.csv.module_name를 맞춰서 "내부 의존"을 찾습니다.

4) internal_edges.csv

실제로 내부 의존으로 매칭된 결과입니다. 의존 그래프의 엣지(간선) 테이블.

internal_edges		
src_file	dst_file	imported_module
Desktop/2026-Spring/cs188/tutorial/grading.py	Library/Mobile Documents/.Trash/cs188 proj 2/multiagent/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	codeql-db/src/Users/yhkim/Desktop/2026-Spring/cs188/cs188-proj/reinforcement/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/Projects/proj1/cs188-sp26-proj1-traceroute/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	codeql-db/src/Users/yhkim/Desktop/2026-Spring/cs188/tutorial/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	codeql-db/src/Users/yhkim/Library/Mobile Documents/.Trash/cs188 proj 2/reinforcement/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/tutorial/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Library/Mobile Documents/.Trash/cs188 proj 2/search/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/cs188-proj/search/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/cs188-proj/reinforcement/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	codeql-db/src/Users/yhkim/Desktop/2026-Spring/cs188/search/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/cs188-proj/tracking/util.py	util
Desktop/2026-Spring/cs188/tutorial/grading.py	Desktop/2026-Spring/cs188/cs188-proj/multiagent/util.py	util

주요 컬럼

src_file: 의존하는 쪽 (import 하는 파일)

dst_file: 의존받는 쪽 (import 당하는 파일)

imported_module: 매칭된 모듈명

해석 팁

한 줄 = "src_file -> dst_file" 의 직접 의존 1개

dst_file 기준으로 개수 세면 "누가 많이 참조되는 파일인지" (중요도 후보)

src_file 기준으로 개수 세면 "내부 의존이 많은 파일인지" (복잡도/결합도 힌트)

5) file_scored.csv (최종)

Git 지표 + 내부 의존 지표를 파일 기준으로 합친 결과입니다.

file_scored											
file	commit_count_180d	authors_180d	churn_lines_180d	last_modified	top1_author	top1_share	ref_count	imports_internal_count	file_type	is_code_file	category
hw03/src/ArrayExercises.java	1	1	14	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/src/ListExercises.java	1	1	33	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/src/MapExercises.java	1	1	27	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/tests/ArrayExercisesTest.java	1	1	55	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/tests/DessertTest.java	1	1	242	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/tests/ListExercisesTest.java	1	1	65	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw03/tests/MapExercisesTest.java	1	1	63	2025-08-29T07:36:43+09:00	dwang@berkeley.edu	1.0	0	0	other	FALSE	NonCode
hw04/src/IntList.java	1	1	119	2025-08-31T22:12:44-07:00	75148537+AngryAny@users.noreply.github.com	1.0	0	0	other	FALSE	NonCode

추가 컬럼(조인/파생)

ref_count: 내부에서 이 파일을 참조하는 수 (internal_edges의 dst_file in-degree)

imports_internal_count: 이 파일이 참조하는 내부 파일 수 (src_file out-degree)

file_type: 파일 유형 휴리스틱 분류

is_code_file: 현재는 Python 코드 파일 여부

category: 최종 카테고리

현재 카테고리 규칙 (Python 파일 기준)

Core: 많이 참조되고(ref_count 높음) 최근 churn도 큼

Stable: 많이 참조되지만 churn은 낮음

DeadCandidate: 참조가 거의 없고 churn도 낮음

Other: 위 조건 아님

NonCode: 현재 코드 분류 대상이 아닌 파일

해석 팁

ref_count 높고 churn 높음 = 변경 영향도 + 변경 빈도 모두 높아서 리스크 관리 대상

ref_count 높고 churn 낮음 = 핵심이지만 안정적인 공용 모듈 가능성

DeadCandidate = 삭제 후보라기보다 "검토 후보" (테스트/리소스일 수 있음)

지금 이 data 폴더 결과가 다소 이상하게 보이는 이유 (중요)

현재 데이터는 서로 다른 분석 대상이 섞여 있습니다.

file_git_metrics.csv: Java 파일 16개 위주

internal_edges.csv: Python 경로 대량(여러 경로 포함) 데이터

그래서 file_scored.csv에서

ref_count = 0이 많고

file_type = other

category = NonCode

로 나옵니다.

즉, 로직이 고장난 게 아니라 "같은 repo를 대상으로 뽑은 데이터가 아님" + "현재 분류기가 Python 위주"인 영향입니다.

imports.csv / modules.csv: CodeQL 원천 추출

internal_edges.csv: 내부 의존 그래프 생성 결과

file_git_metrics.csv: Git 변경/소유 지표

file_scored.csv: 둘을 합친 파일 우선순위/리스크 테이블

FastAPI – Parsing

2/26(목) Usage Tracking 미팅

데이터는 주기적으로 생성
 질문이 들어오면 특정 **extractor** 클래스가 호출
SQL or DataFrame 필터링으로 값 반환

CSV파일로 질문에 대응되는 값들을 어떻게 만들어볼지 고민해보기
 자주 물어보는 질문의 값을 어떻게 제공해줄 수 있는지

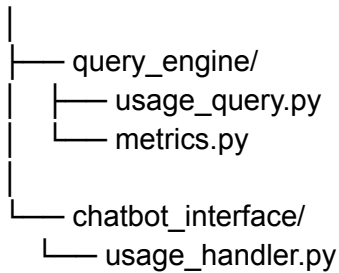
파생 가능한 feature		
commit.csv	Eng Definition	Kor Definition
file_name	File path	파일 경로
commit_count	Total number of commits that modified the file (all-time)	해당 파일을 수정한 전체 커밋 수
added_lines	Total lines added to the file (all-time)	해당 파일에 추가된 전체 라인 수
deleted_lines	Total lines deleted from the file (all-time)	해당 파일에서 삭제된 전체 라인 수
churn_line	added_lines + deleted_lines; total change volume	추가+ 삭제; 합계 변경량
last_modified_date	Latest commit timestamp for the file	파일의 마지막 수정 시각
unique_author_count	Number of distinct authors who modified the file (all-time)	파일 수정에 참여한 고유 작성자 수
author.csv	Eng Definition	Kor Definition
file_name	File path	파일 경로
author_name	Author identity (name/email normalized as you choose)	작성자 식별값(이름/이메일 정규화)
author_commit_count	Number of commits by this author on this file (all-time)	이 작성자가 해당 파일에 남긴 커밋 수
author_line_added	Lines added by this author on this file (all-time)	이 작성자가 해당 파일에 추가한 라인 수
author_line_deleted	Lines deleted by this author on this file (all-time)	이 작성자가 해당 파일에서 삭제한 라인 수
author_churn_line	author_line_added + author_line_deleted	작성자 기준 변경량(추가+ 삭제)
author_share	author_churn_line / churn_line for the same file (0~1)	파일 전체 변경량 대비 작성자 기여 비율(0~1)
message.csv	Eng Definition	Kor Definition
commit_hash	Commit SHA	커밋 고유 해시 (SHA)
file_name	File touched by the commit	해당 커밋이 변경한 파일
author_name	Commit author	커밋 작성자
date	Commit timestamp	커밋 시각
message	Commit message text	커밋 메시지 본문
dependency.csv	Eng Definition	Kor Definition
src_file	File that imports another file/module	다른 파일/모듈을 import하는 파일
dst_file	Internal file being imported	import 대상 내부 파일
imported_module	Imported module path used for mapping	매핑에 사용된 import 모듈 경로
line_num	Import statement line number in src_file	src_file 내 import 구문 라인 번호
ref_count	Number of internal files importing dst_file (derived from dependency edges)	dst_file 를 import하는 내부 파일 수 (집계값)

임시 구성

usage_layer/

```

|
|— data_generation/
|   |— generate_git_features.py
|   |— generate_import_graph.py
|   |— build_usage_dataframe.py
|
|— data_storage/
|   |— usage.db (or CSV files) 확장성을 생각하면 SQL로 구성
  
```



1. DataFrame 생성 스크립트
2. 데이터 저장
3. Chatbot -> 질문 들어옴
4. 특정 클래스 호출 -> 데이터 추출
5. 결과반환

롤베이스 Q&A를 뽑고 그거 바탕으로 데이터분석이나 값

아키텍처

프론트

1 질문

7 답변

<->

백엔드

2 질문 받음

3 이 질문이 롤/LLM 판단

4 IF 롤 -> 롤베이스에 맞는 sql을 찾고 끝 -> sql을 DB서버로 던짐

6 받은 결과를 프론트로 던져줌

<->

DB서버(sql을 수행)

5 받은 sql을 수행하고 결과를 백엔드로 던짐

예시: (저기들어가는 서버 주소랑 mongodb는 예시라 보며됩니다)

1. (프론트) 사용자가 챗봇에 질문한다.
 - 질문 : 이 python 파일 누가 마지막으로 수정했어?
2. (백엔드) 챗봇이 이 질문이 원지 분류(롤베이스 or LLM) 해서 롤베이스 Q&A이다?
 - 롤베이스는 백엔드에서 처리
 - LLM단은 API를 쓰던지... 아니면 LLM 서버는 따로 있을 수 있음
3. (백엔드)특정 소스를 수행한 결과(DB서버)를 뱉는다.
 1. 특정 소스가 DB에서 데이터 가져오는 sql 일꺼다.
 2. python mongodb connection(10.25.11.11 -> 10.25.25.13)을 통해서 q_file_last_modified.sql 해당 결과를 불러와서 뱉어준다.
 3. 백엔드 서버 linux 10.25.11.11

4. DB 서버가 linux (이 안에 mongodb) : 10.25.25.13

2/28 (금) UI/UX + ideation 미팅

1. Extension button -> Explain / Code 두가지 모드로 나누기

a. Explain:

i. 이 파일 설명해줘

1. 파일 선택후 **extension launch**하고 파일 설명해줘 하면 바로 해당 파일 내부 내용 다 설명 해주기
2. 파일과 관련된 구조 - 뭐랑 연결 되어있고, 임포트, 함수 역할
3. 담당자 알려줘 - 누가 마지막으로 커밋했는지, 또는 누가누가 커밋하고 있는지

ii. 이 모듈(function - def) 설명해줘

1. 모듈에 대한 설명을 원할 때에는 모듈 설명해줘 이후 특정 모듈을 스크롤/하이라이트/선택 하고 엔터/액션을 했을 때 자동으로 챗봇에 인식되도록
2. 모듈버튼을 만들어서 모듈 리스트 선택란 뜨는 방법
- 3.

iii. 프로젝트 전체 구조 설명해줘

1. 단순 프로젝트 전체 구조 설명해줘 하면 모든 파일에 대한 코딩을 다 설명해주기
2. 폴더 단위로 설명 → 파일 단위 → 모듈 단위 설명

b. Code (의존성):

i. 이 코드를 바꿨을 때 어디에 영향이 가는지 알려줘 (코드를 선택하는 방식에 대한 연구 필요 - 모듈 설명해줘랑 같으면 좋을듯)

ii. 담은 영향이 가는 파일과 모듈을 리스트 형식으로 보여주고 링크 되기

1. 내가 원하는 파일을 선택하면 그 파일내에 영향이 가는 모듈 하이라이트
2. 내가 원하는 모듈을 선택하면 그 특정 모듈이 우선적으로 보여지도록 화면 조정 및 하이라이트

iii. Primary side bar가 아닌 secondary side bar에 챗봇 구현

1. 파일 이동하다가 내가 하던 모듈하는 사람한테 돌아가고 싶으면 내 채팅/질문에 모듈 누르면 수정하던 곳으로 다시 돌아가기

Stretch Goal

1. 이 코드 Run할 때 걸릴 시간 **estimate** 해주기
2. 이 코드 부분을 고치면 **error**날 수 있는 부분을 미리 AI가 계산 및 파악해서 알림창 같이 알려주기
3. 라이브 문서를 vs code extension으로 할건지 git hub에 보여줄건지. 어디서 어떻게 보여주면 좋을지 아이디어이션 필요하다...
4. Vs 코드 자체적 챗봇과 우리의 차별점 찾아보기 - 어떤 기능들을 가지고 있는지 알아오기 → 이미 있는 기능은 전부는 필요없지만, 우리가 기본적으로 가지면 어느정도는 좋을듯.
5. 사람 답변도 챗봇 안에 인풋으로 넣기
 - a. 호버링도 좋지만, 호버링을 했을 때 계속 뭐가 뜨면 나중에 마우스 움직일때마다 너무 많은 것들이 떠서

- b. Ai가 답변을 하면 시니어가 확인 할 수 있는 시스템
 - i. 신입사원은 질문이 많을 수 있는데 모든 질문을 다 시니어한테 들어가서 확인을 부탁하면 너무 귀찮으니깐 시니어가 확인했으면 하는 특정 질문들에만 태그를 하는게 더 나을 수도?!
- c. 챗봇한테 질문할때 특정 사람 태그 하면 그 사람에게도 질문이 들어가서 나중에 그 사람/시니어가 답변을 하면 ai답변과 사람 답변 비교 하면서 알려줘도 좋을듯
- d. 나중에 같은 질문을 누군가가 했을 때 챗봇 히스토리를 보고 사람이 한 답변을 우선적으로 알려주고 추가적으로 ai가 변화된 답변을 알려줄 수 있게

Things to do:

재운:

- 오늘 이야기 한것까지는 대충이라도 틀 만들어보기
 - 설명할때 사용할 수 있는 레퍼런스용
- **Extension Capability** 읽어보고 **ideation**
- **UI/UX guideline** 읽어보고 **ideation**
- **Ideation** 끝나면 공유 및 남은 **feature**들 디자인 시작

찬영:

- **Extension Capability, testing / publishing** 부분 읽어보기
- 오늘 이야기 나온 부분 구현 가능한지 테스트 해보고 공유하기
 - 안되는 부분은 피그마로 디자인하면 넣을 수 있는지 체크 후 업뎃
 - 가능하면 재운 디자인 시작!

승연:

-

Appendix:

- https://www.google.com/search?q=%EC%BD%94%EB%94%A9+%EC%9D%98%EC%A1%B4%EC%84%B1+%EA%B7%B8%EB%9E%98%ED%94%84&sca_esv=9dfdbda55d993924&udm=2&biw=1470&bih=760&sxsrf=ANbL-n4mSX4JTF7H6heK3CyBYf5C710eng%3A1772264817575&ei=cZ2iaZXtlq3m5NoPpPKGOA&ved=0ahUKEwjV6Zj32PuSAxUtM1kFHSS5AQcQ4dUDCBI&uact=5&oq=%EC%BD%94%EB%94%A9+%EC%9D%98%EC%A1%B4%EC%84%B1+%EA%B7%B8%EB%9E%98%ED%94%84&gs_l=Eqtdnd3Mtd2l6LWltZyla7L2U65SplOydmOyhtOyEsSDqt7jrnpiIIRIIBhQwARY_BZwAHgAkAEAmAEAoAEAggEAuAEDyAEA-AEBmAlAoAlAmAMAIAYBkgcAoAcAsgcAuAcAwgcAyAcAgAgA&sclient=gws-wiz-img#sv=CAMSWBoyKhBILTBwV2NEYzFzem5KTWhNMg4wcFdjRGMxc3puSk1oTToOUVZ3cTBUZDE1d0c2LU0gBCocCgZtb3NhaWMSEGUtMHBXY0RjMXN6bkpNaE0YADABOAAyByD7l9rcCzACSggQAhgCIAloAg

3/4 (수) 혼다 준비 미팅

3/6 혼다 발표 준비

- 슬라이드 링크:
https://www.canva.com/design/DAHC2VCB-4I/oQhYLiopCyM7Ureptru-DA/edit?utm_content=DAHC2VCB-4I&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton
- 1. 프로젝트 전체 구상 - 윤하
 - a. Phase 1: Repo Buddy / Usage Tracking Layer
 - b. Phase 2: Auto-documentation
- 2. Usage Tracking Layer - 종훈 <rule-based>
 - a. ppt
 - i. 슬라이드 추가할 것: 찬영 (챗봇), 승연 (스레드)
 - ii. 슬라이드 수정할 것: 영어로 텍스트 변환(?)
- 3. Repo buddy (UI) - 찬영
 - a. 어떻게 구상했는지? -> 데모까지
 - b. 디자인
- 4. demo 설명 가볍게 - 영어로 고쳐야됨
- 5. 질문 - 관호
 - a. 신입들이 가지고 있는 질문들이 주로 어떤 유형인지? (파싱 후 유의미한 피처를 고르기 위해서) - 데이터 분석을 시도했으나, 의미가 없다고 판단 / 룰 베이스로 하는 것을 고려함
 - b. 기본적인 질문 리스트 (FAQ) / 질문 작성 시 키워드
 - c. DB & Server
 - i. Usage Tracking Layer: 왜 db 가 필요했는지..?
 - ii. Repo Buddy: 개인이 혼자 개발하는 환경에서는 현재 작업물을 활용해서 구글닥이나 슬랙에서 사용하는 코멘팅이나 디스커션 등의 기능들을 로컬 환경에서 사용할 수 있음. 팀 콜라보의 경우, db 와 server 가 있어야 서로 같은 정보를 주고 받을 수 있을텐데... 이 부분에 대한 인사이트? 피드백?

혼다 미팅 상관없이 확인해 볼 것

- 개별적으로 DB & Server 공부 및 리서치
- DB: Supabase
- Server: Vercel
 - 서버 무료 플랜으로 시작하는데... 유저/리포 크기/사용량 등에 따라 사용료가 얼마나 될지

(금) Ryan 미팅 이 후 미팅 – 3/6(Fri) @1700
(토) 11시 이후 – 3/7(Sat) @1500

Usage Tracking Layer DB 다 같이 고민

Supabase - DB

Vercel - Hosting (Server) ㄱ

3/6 (ㄹ) Meeting w/ Honda

Shared Google Drive from Honda (OpenAI API Key, etc):

https://drive.google.com/drive/folders/1IU9Y3ZqK-Hpz8_i9UXpH14IQZ0ymExgs?usp=sharing

QnAs

1. What questions do newbies ask?
 - a. You don't know what you don't know - Relying on FAQs at the development level is not ideal
2. Guidance on db/server setup?
 - a. Don't spend any money
 - i. Open-source project doesn't have to support full storage, db, or server
 - b. Think of this project as a new approach to version control
 - i. New generation of GitHub or Git

제공된 소스(회의 녹취록)를 바탕으로 회의의 전체 내용과 핵심 내용을 정리해 드립니다.

1. 전체 내용 요약

이 회의는 학생 개발 팀이 진행 중인 코드 저장소 분석 및 AI 채팅 도구(**Repo Chat**) 프로젝트의 1단계(**Phase 1**) 진행 상황을 공유하고, 혼다(**Honda**) 측 담당자인 라이언(**Ryan**)에게 피드백과 조언을 구하는 자리입니다. 팀은 코드 사용 패턴을 추적하는 **Usage Tracking** 레이어와 **VS Code** 확장 프로그램 기반의 **UI** 개발 현황을 보고했으며, 인프라 구축(데이터베이스, 서버) 및 온보딩 질문 유형에 대한 어려움을 논의했습니다.

2. 핵심 내용 정리

가. 기술적 진행 상황 (**Usage Tracking & UI**)

Usage Tracking 레이어: 코드 변경 이력과 개발 활동을 기반으로 ****이력(History)**, **소유권(Ownership)**, **의존성(Dependency)**, **위험도(Risk)****라는 네 가지 주요 측면을 분석합니다. 시스템 설계 변경: 단순 데이터 분석에서 벗어나 실시간 답변이 가능한 ****데이터 엔지니어링 중심의 시스템 설계(SQL 데이터베이스 기반)****로 전환했습니다.

UI 및 기능: VS Code 확장 프로그램 내에서 **AI** 채팅을 통해 전체 구조, 파일, 함수에 대한 설명을 제공하며, 특정 파일 수정 시 영향을 받는 파일을 식별하는 기능을 구축했습니다.

나. 주요 질문 및 당면 과제

신입 엔지니어의 질문 유형: 팀은 신입 엔지니어가 주로 묻는 질문(**FAQ**)을 파악해 시스템에 반영하고자 했으나, 라이언은 현재 정해진 **FAQ**는 존재하지 않는다고 답했습니다.

인프라 구축의 한계: 팀은 협업을 위해 공유 데이터베이스와 서버가 필요하다고 제안했으나, 라이언은 비용 문제와 기업 내부 보안 절차를 이유로 로컬 **SQLite**를 사용하는 '**개념 증명(Proof of Concept)**' 수준에 머물 것을 권고했습니다.

다. 라이언(**Honda**)의 피드백 및 가이드라인

비용 및 모델 사용: 학생들은 비용을 지불하지 않아야 하며, 라이언이 제공한 공유 드라이브의 **OpenAI API** 키를 사용하되 비용 효율적인 '**mini**' 모델(**GPT-4o mini** 등) 사용을 권장했습니다.

오픈 소스 전략: 프로젝트를 오픈 소스로 릴리스할 때는 **API** 키나 데이터베이스를 포함하지 않고, 사용자가 자신의 키를 입력해 로컬에서 실행할 수 있는 '**스켈레톤**' 구조로 만들어야 합니다. x

미래 비전: 라이언은 이 프로젝트를 **버전 관리 지능화(Version Control Intelligence)**의 첫 단계로 평가했습니다. 현재의 **GitHub**이 레코드판 수준이라면, **LLM**과 벡터 데이터베이스를 결합한 이 도구가 차세대 지능형 버전 관리 시스템이 될 가능성이 있다고 격려했습니다.

비교 대상: **Google**의 **NotebookLM**이 **PDF**나 문서를 분석하듯, 이 프로젝트는 코드 저장소에 특화된 유사한 가치를 제공할 수 있다고 언급했습니다.

라. 기타 지원 사항

라이언은 학생들에게 **LinkedIn** 연결, 추천서 작성 등 커리어 관련 지원을 아끼지 않겠다고 약속했습니다.

API 키 보안을 위해 **.env** 파일에 저장하고 **.gitignore**에 추가하여 **GitHub**에 노출되지 않도록 주의할 것을 당부했습니다.

이 보고서는 제공된 소스에 근거하여 작성되었습니다. 추가적인 분석이나 특정 부분에 대한 상세 내용이 필요하시면 말씀해 주세요.

SQLite

3/9 (월) GM

생각해야 할 부분들

1. 로컬에서 돌아가는 Repo Onboarding Buddy

- > 사용자로부터 LLM API를 받아 구현한다고 하면, API를 입력받았을 때 처리되는 과정은 어떻게 구현되는가? 사용자가 자신의 API key와 LLM 모델을 입력했을 때, 우리 Repo Buddy extension이 repo 정보를 활용해서 해당 LLM을 호출하고 답변을 생성하는 구조를 어떻게 설계해야 할까? 즉 각각 달리 입력된 LLM을 활용할 방법을 어떻게 구현할 것인지? (code-base or prompt-base?)
- > 로컬 DB를 구현할 수 있을까? / SQLite로 로컬 DB를 구현할 수 있을 것 같은데 실제로 가능한지?
- > 아예 DB를 사용하지 않는다면 어떻게 구현할 수 있을까?
- > 이게 로컬에서만 돌아간다면 과연 Cursor와 같은 소프트웨어 툴과 무슨 차이인지? 어차피 Cursor도 폴더 및 파일 분석 다 하는데 어떤 차별점을 둘 수 있을까?

2. 기업 / 팀 프로젝트(또는 오픈소스)에서 돌아가는 Repo Onboarding Buddy

- > 기업 / 팀 프로젝트에서는 서버를 동반한 DB가 필요할 텐데, DB 지원 없이 final product를 구현할 수 있을까? 아니면 그냥 Supabase free trial 같은거를 써서 좀 더 발전된 MVP를 만들어야 할지?
- > 아예 오픈 소스를 타겟으로 하자니 오픈 소스에서 코드를 가져온다고 했을 때 과연 누가 이걸 commit했는지 관심이 있을까? (ex. fastAPI 코드를 가져올 때 누가 이 module을 구현했는지는 별로 안 궁금할 수 있음)

-> 제가 중간에 든 생각은 로컬에서 돌아가는 Repo Onboarding Buddy를 만들되 github에서 pull할 때 코드 뿐만 아니라 git 히스토리 데이터를 가지고 와 github 관련 데이터들도 분석할 수 있으면 비록 로컬에서 돌아가는거긴 하지만 마치 github에 올라와있는 코드를 분석해주는 것처럼 구현할 수 있지 않나 싶습니다. (물론 실시간 분석은 불가능하지만 사용자가 pull하면 그때 바로 분석하는 거로 구현)

Stretch Goal

☐ Github UI version

- Github command들을 UI로 보기 쉽게, 사용하기 쉽게 구현하자는 것이 취지인데 그럼 타켓이 결국 코딩에 익숙하지 않은 일반인이다. 근데 그런 일반인들 중 코드를 짜고 Github에 올리려 하는 사람들이 많을까?
- 우리가 만들고자 했던 Repo Onboarding Buddy와 무슨 연관이 있을까..?

3/14 (토) 미팅

Git과 GitHub의 역할

Git이란?

Git은 **버전 관리 시스템 (Version Control System)**입니다.

즉, **파일과 코드의 변경 기록을 관리하는 도구**입니다.

Git을 사용하면 다음과 같은 것을 추적할 수 있습니다.

- 코드가 **언제** 수정되었는지
- **누가** 수정했는지
- **무엇이** 변경되었는지
- **왜** 수정되었는지

이를 통해 프로젝트의 변경 이력을 체계적으로 관리할 수 있습니다.

Git의 핵심 기능

1. 버전 관리

Git은 코드의 모든 변경 기록을 저장합니다.

따라서 문제가 발생했을 때 **이전 버전으로 언제든지 되돌아갈 수** 있습니다.

2. 코드 변경 추적

어떤 코드가 **어떻게** 변경되었는지 비교할 수 있습니다.

예를 들어

- 어떤 코드가 삭제되었는지
- 어떤 코드가 새로 추가되었는지

를 확인할 수 있습니다.

3. 수정자 확인

특정 코드가 **누구에** 의해 수정되었는지 확인할 수 있습니다.

이는 특히 **팀 프로젝트에서 중요한 기능**입니다.

4. 브랜치 (Branch)

브랜치는 ****개발 작업을 분리하는 기능****입니다.

예시 구조

...

main

└ feature-login

└ feature-payment

└ bugfix-auth

...

각 기능을 ****독립적으로 개발한 뒤 나중에 하나로 합칠 수 있습니다.****

GitHub란?

GitHub는 ****Git 저장소를 온라인에서 관리하고 협업할 수 있게 해주는 플랫폼****입니다.

Git이 ****버전 관리 시스템 자체****라면

GitHub는 ****그 저장소를 인터넷에서 관리하는 서비스****라고 볼 수 있습니다.

GitHub의 주요 기능

1. 협업

여러 명의 개발자가 같은 프로젝트에서 동시에 작업할 수 있습니다.

2. Pull Request

코드를 바로 합치는 것이 아니라

****코드를 검토한 뒤 병합(Merge)하는 과정****을 제공합니다.

3. 코드 리뷰

팀원이 코드 품질을 확인하고 개선 의견을 제시할 수 있습니다.

4. 이슈 관리

버그나 해야 할 작업을 **Issue** 형태로 관리할 수 있습니다.

5. 프로젝트 관리

작업 진행 상황을 **보드** 형태로 관리할 수 있습니다.

Target User

이 프로젝트의 주요 타겟은 **개발자가 아닌 일반 사용자**,
특히 **AI 코딩 도구를 사용하는 바이브 코더 (Vibe Coder)**입니다.

이들은 코드를 작성할 수는 있지만
Git과 같은 개발 도구에는 익숙하지 않은 경우가 많습니다.

해결해야 할 문제

GitHub를 사용하려면 일반적으로 다음 과정을 거쳐야 합니다.

내 파일 → Git 명령어 → GitHub

즉, 사용자는 **Command Line**에서 Git 명령어를 사용해야 합니다.

예를 들면

...

```
git add
git commit
git push
...
```

이 과정은 개발자에게는 익숙하지만
일반 사용자에게는 **높은 진입 장벽**이 됩니다.

결과적으로

- GitHub 사용이 어렵게 느껴지고
- Git 자체가 ****접근 장벽****이 됩니다.

해결책

이 문제를 해결하기 위해

****Git 명령어를 직접 사용하지 않아도 되는 UI/UX 인터페이스를 제공합니다.****

즉,

Git을 제거하는 것이 아니라

****Git의 복잡한 과정을 사용자에게 보이지 않게 추상화합니다.****

기존 구조

나의 파일 → Command(Git) → GitHub

개선된 구조

나의 파일 → UI 기반 앱 → GitHub

사용자는 ****클릭 기반 인터페이스****를 통해

- 변경 기록 저장

- 버전 관리

- GitHub 업로드

등을 수행할 수 있습니다.

추가적으로 얻을 수 있는 이점

AI 코딩 도구를 사용하는 사용자들은

GitHub 작업 또한 ****AI Agent를 통해 수행하는 경우가 많습니다.****

이 경우 다음과 같은 문제가 발생합니다.

- 불필요한 ****API 호출****
- 증가하는 ****토큰 사용량****
- 느린 작업 속도

만약 **Git** 작업을 ****로컬 UI에서 직접 처리****할 수 있다면

- 불필요한 **AI** 호출을 줄이고
- 토큰 사용량을 감소시키며
- 더 빠른 작업 흐름을 만들 수 있습니다.

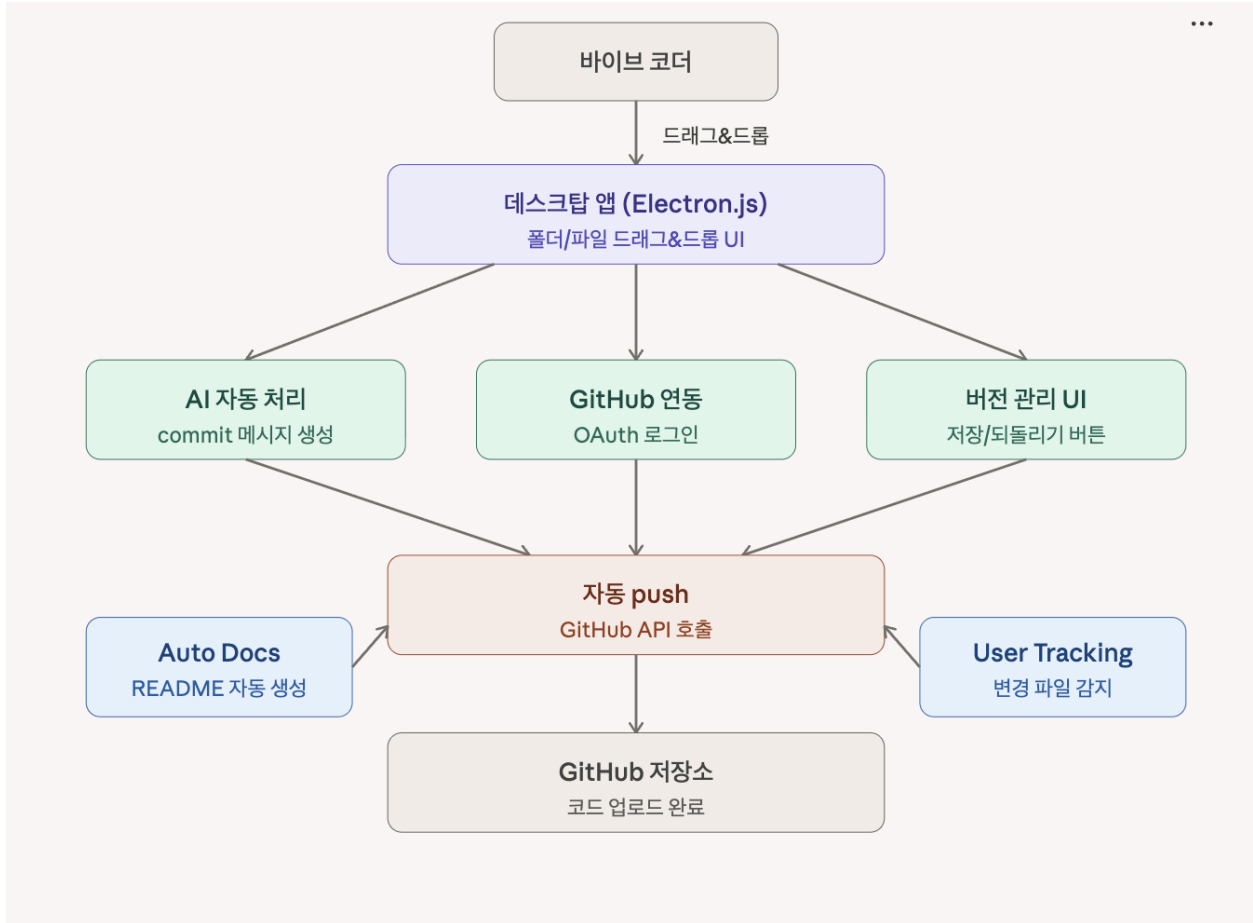
한 문장 요약

이 프로젝트의 목표는

- > **Git**의 복잡한 **Command Line** 과정을 **UI**로 추상화하여
- > 일반 사용자도 **GitHub**를 쉽게 사용할 수 있도록 만드는 것입니다.

참고: 비슷한 서비스는 존재 **but** 개발자 기준 **ui**로 되어있음 그렇기에 더더욱 **git**을 아예 몰라도 사용할 수 있게 해주는 시스템에 초점을 맞춰야될 것 같습니다.

개인적 사건으로는 이번에는 **Ryan**한테 우리의 확실한 타겟이랑 목적에 대해서 설명하고 이번에는 실행쪽의 조언을 구체적으로 얻겠다(?) 라는 느낌으로 미팅을 진행하면 좋지 않을까 합니다.



이 앱은 충분히 구현 가능하다. 다만 “드래그 앤 드롭 업로드” 자체보다 중요한 건, 그 뒤에서 **Git** 상태를 안전하게 만들고 **GitHub**와 정확히 동기화하는 구조다.

구현을 위해 꼭 필요한 것은 크게 7개다.

1. 데스크톱 앱 껍데기
2. **GitHub** 로그인/인증
3. 로컬 파일 스캔
4. 변경 감지
5. **Git** 작업 엔진
6. **GitHub API** 연동

7. 오류 복구 및 안전장치

그리고 구현 방식은 두 갈래가 있다.

- **A안: 로컬 Git 사용**
 - 내부적으로 `git init`, `git add`, `git commit`, `git push`
 - 구현이 더 쉽고 안정적
- **B안: GitHub REST API만 사용**
 - `blob/tree/commit/ref`를 직접 생성
 - Git 설치 없이 가능하지만 구현이 더 복잡

현실적으로는 **A안**으로 **MVP**를 만들고, 필요하면 나중에 **B안**을 추가하는 게 맞다.

2. 근거 및 논리 (Rationale & Logic)

이 앱의 본질이 무엇인가

이 앱은 단순 업로더가 아니다.
실제로는 다음 문제를 감춘다.

- 사용자는 Git 명령어를 모른다
- 사용자는 GitHub 인증 흐름을 모른다
- 사용자는 `commit/push` 개념이 익숙하지 않다
- 사용자는 “기존 `repo`를 덮어써도 되는지” 판단하기 어렵다

즉 이 앱의 핵심은

Git의 복잡함을 **UI** 뒤로 숨기는 것

이다.

그래서 백엔드는 단순 파일 업로드 서버처럼 만들면 안 되고,
“파일 → 변경사항 → **commit** → **push**” 흐름을 정확히 관리해야 한다.

3. 구현을 위해 필요한 것들 (What You Need)

3-1. 앱 형태 결정

이 앱은 웹앱보다 데스크톱 앱이 더 적합하다.

이유는 간단하다.

- 로컬 폴더 접근이 필요함
- drag & drop 폴더 처리가 필요함
- Git 실행이 필요할 수 있음
- 파일 해시 계산, 임시 작업 디렉토리 관리가 필요함

Tauri는 데스크톱 앱 프레임워크이고, 파일 시스템 플러그인을 통해 로컬 파일 접근을 제공한다.

Electron도 native drag & drop과 데스크톱 앱 구성이 가능하다.

추천:

초기 MVP는 **Tauri + React** 또는 **Electron + React**.

내 판단으로는:

- 가볍고 배포 파일 작게 가고 싶으면 Tauri
- Node 생태계로 빨리 구현하고 싶으면 Electron

3-2. GitHub 인증

GitHub REST API를 쓰려면 인증 토큰이 필요하다. GitHub 공식 문서도 REST API 요청에는 적절한 권한의 인증 토큰이 필요하다고 설명한다.

여기서 선택지는 보통 두 개다.

- Personal Access Token 직접 입력
- OAuth 로그인

비전문가 대상이면 **OAuth** 로그인 방식이 더 자연스럽다.
 사용자는 “GitHub로 로그인” 버튼만 누르면 된다.

백엔드적으로 필요한 것:

- GitHub OAuth 앱 등록
- redirect URI 설정
- access token 저장
- 토큰 만료/재인증 처리

여기서 중요한 점은 토큰을 평문 파일에 저장하면 안 된다는 것이다.
 운영체제 키체인이나 **secure storage**를 써야 한다.

3-3. 로컬 파일 스캔

드래그 앤 드롭으로 폴더를 받으면, 앱은 내부적으로 이런 정보를 만들어야 한다.

- 루트 폴더 이름
- 전체 파일 목록
- 상대 경로
- 파일 크기
- 확장자
- 바이너리 여부
- 해시값

예를 들면 내부 구조는 이런 식이다.

```
[
  {
    "path": "src/App.tsx",
    "size": 2841,
    "sha256": "abc123...",
    "binary": false
  },
  {
```

```
"path": "public/logo.png",  
"size": 91823,  
"sha256": "def456...",  
"binary": true  
}  
]
```

이 단계가 중요한 이유는,
나중에 무엇이 새 파일이고, 무엇이 수정 파일이고, 무엇이 삭제 파일인지 판단해야 하기 때문이다.

3-4. 변경 감지

이 앱이 한 번 업로드하고 끝나는 게 아니라면,
반드시 **change detector**가 필요하다.

기본 원리:

- 현재 로컬 파일 해시 계산
- 이전 업로드 시점 스냅샷과 비교
- **added / modified / deleted** 계산

예:

added:

- src/Login.tsx

modified:

- src/App.tsx

- README.md

deleted:

- old/test.js

이게 있어야

- 불필요한 전체 업로드 방지
- 자동 **commit message** 생성

- “정말 이 파일을 삭제할 건가요?” 같은 안전 UI

를 만들 수 있다.

3-5. Git 작업 엔진

이 부분이 핵심이다.

GitHub에 코드를 올리는 방법은 두 가지다.

방법 **A.** 로컬 **Git** 사용

앱이 내부적으로 **Git** 명령을 대신 실행한다.

흐름:

1. 작업 폴더 복사 또는 지정
2. `git init` (필요 시)
3. `git remote add origin`
4. `git add .`
5. `git commit -m "..."`
6. `git push`

이 방식의 장점은 구현이 단순하고, **Git** 동작이 검증되어 있다는 점이다.

방법 **B.** **GitHub API**만 사용

GitHub는 **Git database REST API**를 통해 **raw Git objects**를 다룰 수 있다. **Git blob, tree, commit, reference**를 읽고 쓸 수 있다고 공식 문서가 설명한다.

즉 내부적으로는 다음을 해야 한다.

파일 내용 → **blob** 생성

blob들 → **tree** 생성

tree → **commit** 생성

branch ref → 새 **commit**으로 업데이트

이건 가능하지만, **MVP** 단계에서 굳이 어렵게 할 이유는 적다.

현실적 추천:

처음에는 로컬 **Git** 기반으로 구현.

이게 안정적이고 개발 속도도 빠르다.

3-6. GitHub API 연동

GitHub REST API로 할 수 있는 핵심 작업은 이미 공식적으로 제공됩니다.

- 인증된 사용자 repo 생성
- 파일 내용 create/update/delete
- Git database 객체 생성(blob/tree/commit/ref)

그래서 앱은 최소한 다음 API 흐름 중 하나를 선택하면 된다.

간단한 흐름

새 repo 생성 → 로컬 Git push

API-only 흐름

새 repo 생성 → blob/tree/commit/ref 직접 생성

MVP에서는 첫 번째가 훨씬 낫다.

3-7. 안전장치

이 앱이 진짜 제품처럼 보이려면 안전장치가 필수다.

예상되는 문제:

- 이미 같은 이름의 repo 존재
- 기본 브랜치 이름이 main인지 master인지 다름
- push 권한 없음
- 인증 토큰 만료
- 대용량 파일 포함
- 이미 remote에 다른 커밋이 있음
- 같은 파일명이지만 내용이 달라 충돌

GitHub는 50 MiB 초과 파일에 경고가 발생하고, 100 MiB에서 강하게 제한하며, 큰 파일은 Git LFS 사용을 권장한다. 또한 push size 제한과 object size 제한도 안내한다.

즉 앱에는 최소한 아래 기능이 있어야 한다.

- 업로드 전 파일 크기 검사
 - 대용량 파일 차단 또는 LFS 안내
 - overwrite/새 브랜치 선택
 - 업로드 전 미리보기
 - 실패 시 rollback 또는 재시도
-

4. 추천 구조 (Recommended Architecture)

전체 구조

[Desktop UI]

- └ Drag & Drop View
- └ Repo Settings View
- └ Upload Preview View
- └ Progress / Error View

[Application Core]

- └ Auth Manager
- └ File Scanner
- └ Change Detector
- └ Commit Planner
- └ Git Engine
- └ GitHub Client
- └ Local State Store

각 모듈 역할

Auth Manager

- GitHub 로그인
- access token 저장

- 현재 사용자 확인

File Scanner

- 폴더 재귀 순회
- 숨김 파일 처리
- `.gitignore` 반영 여부 결정
- 파일 메타데이터 생성

Change Detector

- 이전 업로드 기록과 비교
- added/modified/deleted 계산

Commit Planner

- 어떤 파일을 어떤 commit으로 올릴지 결정
- commit message 자동 생성

Git Engine

- 로컬 Git 명령 실행
- repo init / add / commit / push 담당

GitHub Client

- repo 생성
- 원격 repo 존재 여부 확인
- 사용자 repo 목록 조회

Local State Store

- 마지막 업로드 기록 저장
 - repo 연결 정보 저장
 - 업로드 설정 저장
-

5. 가장 현실적인 구현 방법 (Best Practical Implementation)

추천 구현안: 데스크톱 앱 + 로컬 **Git + GitHub API**

이 방식이 제일 현실적이다.

스택 예시

- 프론트엔드: React
- 데스크톱 셸: Tauri 또는 Electron
- GitHub API 클라이언트: Octokit 계열 또는 직접 REST 호출
- Git 실행: simple-git 또는 child process로 git 명령 실행
- 로컬 저장: SQLite 또는 JSON
- 보안 저장: OS keychain

동작 흐름

1단계. 사용자 로그인

- GitHub OAuth 로그인
- access token 저장

2단계. 폴더 드래그 앤 드롭

- 폴더 구조 읽기
- 파일 목록 생성

- 대용량 파일 검사

3단계. 업로드 대상 결정

- 새 repo 생성인지
- 기존 repo 연결인지
- 브랜치를 새로 만들지

4단계. 업로드 계획 생성

- 어떤 파일이 추가/수정/삭제되는지 계산
- commit message 생성
- 사용자에게 미리보기 표시

5단계. Git 실행

- 임시 작업 디렉토리 생성
- 파일 복사
- git init / remote add / add / commit / push

6단계. 결과 표시

- 성공 시 repo URL 보여주기
- 실패 시 원인과 복구 안내

6. 구현 순서 (Build Order)

Phase 1: 최소 MVP

이 단계에서는 “된다”를 증명하면 된다.

필수 기능:

- GitHub 로그인
- 폴더 drag & drop
- 새 repo 생성
- 초기 commit
- push
- 성공 URL 표시

이 단계에서는 기존 repo 업데이트는 아예 빼도 된다.

Phase 2: 실제 사용 가능 수준

그다음 붙일 것:

- 기존 repo 연결
- 변경 감지
- 자동 commit message
- 업로드 전 미리보기
- 대용량 파일 경고
- 오류 복구

Phase 3: 차별화 기능

여기서부터 제품성이 생긴다.

- `.gitignore` 자동 제안
- README 자동 생성
- 프로젝트 타입 감지

- “React app”, “Python app”, “static website” 자동 분류
 - GitHub Pages / 배포 연결 보조
 - 한 번의 클릭으로 **Idea** → **Repo** 완성
-

7. 구현 예시 흐름을 더 쉽게 풀면

사용자가 앱에 폴더를 넣는다.

앱은 내부적으로 이렇게 생각한다.

1. 이 폴더에 뭐가 들어 있지?
2. 올리면 안 되는 큰 파일이 있나?
3. 새 **repo**를 만들까, 기존 **repo**에 넣을까?
4. **commit** 메시지는 뭘로 하지?
5. **Git**으로 묶어서 **GitHub**에 올리자
6. 성공하면 링크를 보여주자

즉 사용자는 “드래그 앤 드롭”만 보지만,
백엔드는 사실상 간소화된 **Git assistant**처럼 동작한다.

8. 리스크 및 보완 포인트 (Risks & Mitigation)

리스크 **1. GitHub Desktop**과 너무 비슷해질 수 있다

이건 기술 문제가 아니라 제품 문제다.

보완 방향은 명확하다.

- “업로드”만 하지 말고
- 비전문가용 초기 세팅 자동화까지 해야 한다

예:

- **README** 자동 생성

- **.gitignore** 자동 추천
 - 프로젝트 타입 감지
 - 업로드 전 쉬운 설명 UI
-

리스크 2. **Git** 충돌을 너무 단순화하면 사고 난다

비전문가 대상이기 때문에 더 위험하다.

보완:

- 기본 정책을 보수적으로
 - 기존 **repo** 덮어쓰기 대신 새 브랜치 권장
 - 삭제 파일은 반드시 확인창
 - **force push** 금지
-

리스크 3. 대용량 파일 때문에 실패할 수 있다

GitHub는 큰 파일과 큰 **push**에 제한을 둔다.

보완:

- 업로드 전 선검사
 - 50 MiB 이상 경고
 - 100 MiB 이상 차단
 - 필요 시 **Git LFS** 안내
-

리스크 4. “토큰 절약”만으로는 가치가 약할 수 있다

이건 솔직히 약한 메시지다.

진짜 가치로 바뀌어야 한다.

- CLI 진입장벽 제거
 - GitHub 업로드 스트레스 감소
 - 초보자의 첫 repo 생성 단순화
 - “로컬 파일 → 공개 repo” 전환 자동화
-

9. 추가 제안 (Further Suggestions)

네가 지금 바로 팀에서 정해야 하는 것은 딱 세 가지다.

첫째

MVP에서 기존 **repo** 업데이트까지 할지, 새 **repo** 생성만 할지

내 판단:

처음엔 새 **repo** 생성만 하라.

기존 **repo** 업데이트는 충돌 처리 때문에 복잡도가 급상승한다.

둘째

Git 엔진을 로컬 **Git**으로 갈지, **GitHub API-only**로 갈지

내 판단:

처음엔 로컬 **Git**으로 가라.

개발 속도와 안정성이 훨씬 낫다.

셋째

우리 앱의 진짜 한 줄 가치가 뭔지

추천 문장:

코딩에 익숙하지 않은 사용자가 로컬 프로젝트를 **Git** 명령어 없이 **GitHub**에 올릴 수 있게 해주는 데스크톱 앱

이 문장이 있어야 기능 선택이 쉬워진다.

3/16 (월) 미팅

추가할 만한 **Git command**들

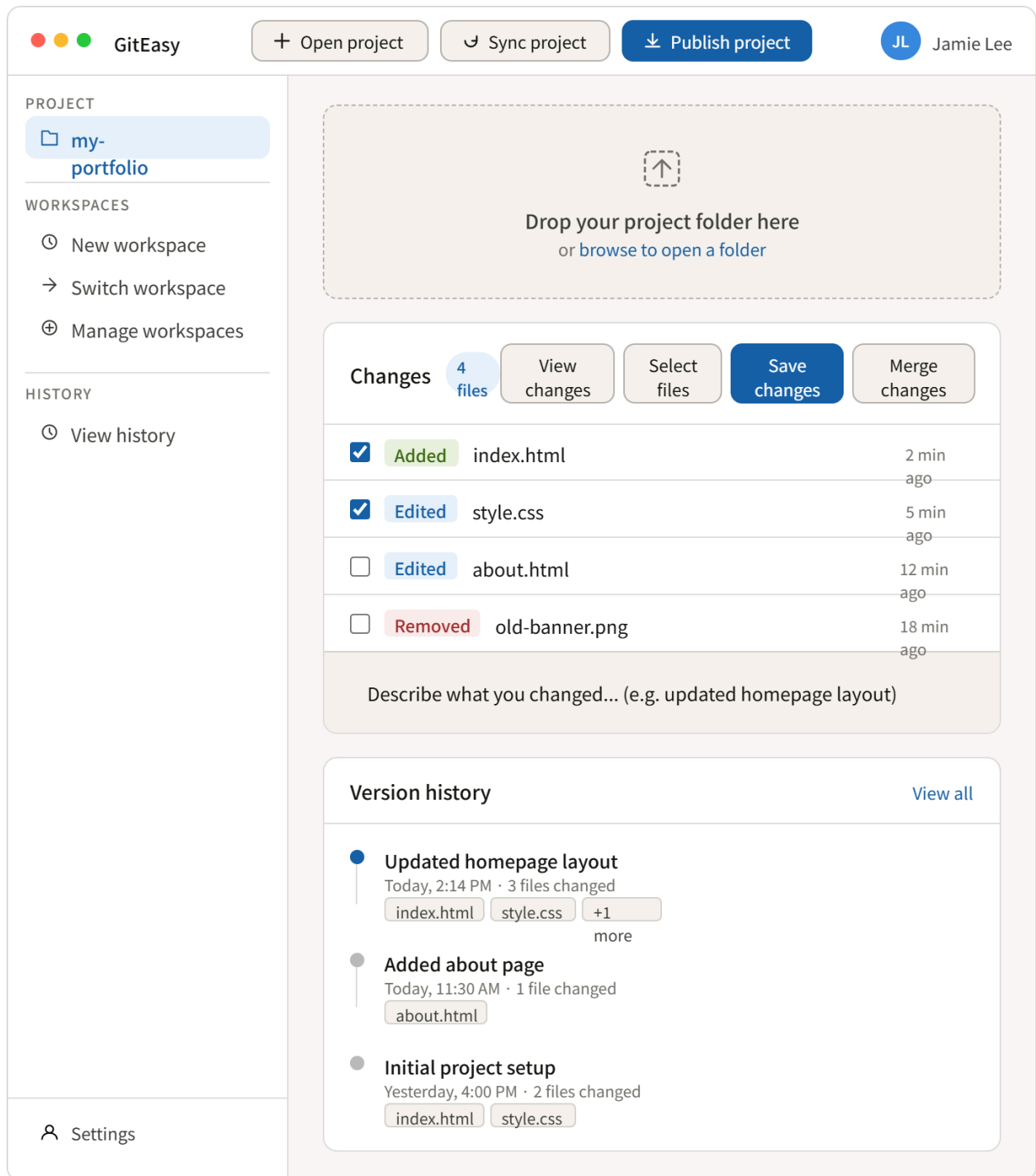
1. git clone → Download Project
2. git status → Check My Changes
3. git add . / git add file (전체 업데이트할 필요없이 단일 파일로도 업로드가능하게) → Select Files to Save
4. git commit -m "" → Save Changes
5. git push → Upload to GitHub
6. git pull → Get Latest Version
7. git branch → Create/View Workspace
 - i) git switch / git checkout → Switch Workspace
 - ii) git switch -c -> Create&Switch to that workspace
8. git merge → Combine Changes
9. git log → View Save History

UI 팀 (3명) – Figma

- 1) Buttons: 자연어로
- 2) drag-and-drop
- 3)

Backend 팀 (3명)

UI로 구성한다면 이런 느낌



[Git_GUI_Tools_Comparison] 시장 조사

<https://docs.google.com/spreadsheets/d/1C32--depqukgVT225Rbd6kii2NQ6Ap3LvHKBJNs7hCQ/edit?usp=sharing>

Ryan 발표

1. 프로젝트 변경 이유 – what you said, we completely understand the barrier that we've met and discussed. (DataBase, 토큰소모)
2. 시장성 조사 / 차별점
3. UI – Figma w/ MVP
4. Backend 데모 보여주기
5. We'd like to finalize with this project setup. Advise? Add-ons?

3/17 (화) git/github 프로젝트 설명

\$

Git/GitHub UI Abstraction Project — Brainstorming Summary

1. 프로젝트의 출발점

우리가 처음 해결하고자 한 문제는 단순히 “Git이 어렵다”가 아니다.

실제로는 많은 사용자, 특히 비개발자나 초보 사용자가 아래와 같은 어려움을 겪는다.

- Git 명령어를 이해하기 어렵다
- **command** 창 사용 자체가 부담스럽다
- 로컬 파일을 **GitHub**에 올리는 과정이 복잡하다
- 실수했을 때 어떻게 복구해야 하는지 모르겠다
- **GitHub**와 로컬 프로젝트의 관계를 직관적으로 이해하기 어렵다

즉, 문제의 핵심은 다음과 같다.

- > 사용자는 파일을 다루듯 프로젝트를 관리하고 싶지만,
- > 실제로는 **Git command**와 **GitHub workflow**를 먼저 배워야 한다.

2. 우리가 만들고 싶은 것

우리가 만들고 싶은 것은 **Git**을 없애는 제품이 아니다.

우리가 만들고 싶은 것은 다음과 같은 제품이다.

- > 사용자가 **command** 창에서 **Git** 명령어를 직접 입력하지 않아도,
- > **UI** 기반 앱을 통해 로컬 프로젝트를 관리하고 **GitHub**와 쉽게 연결할 수 있게 해주는 도구

즉, 핵심은:

- **Git command workflow**를
- **UI workflow**로 바꾸는 것

예를 들어 기존에는 사용자가 아래처럼 작업했다.

```
``bash
git add .
git commit -m "save"
git push
``
```

우리가 원하는 경험은 아래와 같다.

- 파일 **drag and drop**
- 변경 내용 확인
- 버튼 클릭으로 저장
- 버튼 클릭으로 **GitHub** 업로드
- 버튼 클릭으로 이전 상태 복구

즉 사용자는 “**Git**을 쓰고 있다”는 느낌보다

> “파일을 관리하듯 프로젝트를 다루고 있다”

는 느낌을 받아야 한다.

3. 프로젝트의 핵심 가치

이 프로젝트의 핵심 가치는 크게 세 가지로 정리할 수 있다.

1. 진입장벽 완화

Git 명령어와 **terminal** 사용에 익숙하지 않은 사용자도 쉽게 접근할 수 있다.

2. 사용자 친화적 워크플로우

Git 개념을 직접 노출하지 않고도 프로젝트 저장, 복구, 업로드를 직관적으로 수행할 수 있다.

3. 로컬 **Git** + **GitHub** 연동성 강화

로컬에서 작업한 내용을 자연스럽게 **GitHub**와 연결하여 관리할 수 있다.

4. 우리가 논의한 방향들

브레인스토밍 과정에서 프로젝트 방향은 크게 세 가지 가능성으로 나뉘었다.

방향 A. **Git-less** 시스템

Git 없이 자체적인 저장/복구 시스템을 만드는 방향

장점

- 완전히 독립적인 제품 설계 가능
- UX를 자유롭게 구성 가능

단점

- Git의 핵심 기능을 새로 구현해야 함
- 저장 구조, 복구, diff, versioning 등을 직접 설계해야 함
- 구현 난이도가 매우 높음

방향 B. Snapshot 기반 로컬 안전 시스템

프로젝트 상태를 snapshot처럼 저장하고 복구 중심으로 설계하는 방향

장점

- 사용자가 이해하기 쉬운 구조
- 복구/실험 중심 UX 설계 가능

단점

- naive하게 구현하면 저장 용량이 많이 늘어날 수 있음
- Git commit과 snapshot이 동시에 존재하면 source of truth가 두 개가 되는 문제 발생
- 프로젝트의 중심이 GitHub 연동보다 로컬 복구 쪽으로 치우칠 수 있음

방향 C. Git UI Abstraction Tool

Git은 내부 엔진으로 사용하되,
사용자는 **Git command**를 직접 보지 않고 **UI**로만 작업하는 방향

장점

- 가장 현실적인 구현 가능성
- Git의 안정성과 효율성을 그대로 활용 가능
- 사용자는 **command**를 몰라도 됨
- GitHub 연동까지 자연스럽게 연결 가능

단점

- 내부적으로는 **Git** 의존성이 존재함
- “**Git-less**”라고 부르기에는 부정확할 수 있음

5. 현재 가장 적합한 방향

지금까지의 논의를 종합하면,
현재 프로젝트 목적에 가장 잘 맞는 방향은 ****방향 C****이다.

즉,

- > Git을 제거하는 것이 아니라,
- > **Git command** 기반 **workflow**를 **UI** 기반 **workflow**로 추상화하는 방향

이 프로젝트는 결국 다음과 같이 정의하는 것이 가장 적절하다.

- > 비개발자나 초보 사용자도 **Git** 명령어 없이
- > 로컬 프로젝트를 관리하고 **GitHub**와 쉽게 연결할 수 있도록 돕는 사용자 친화적 개발 관리 도구

6. 내부 구조에 대한 논의

6.1 Git을 내부 엔진으로 사용할 것인가

많은 논의 끝에, **Git**을 내부 엔진으로 사용하는 것이 가장 현실적이라는 결론에 가까워졌다.

이유는 다음과 같다.

- **Git**은 이미 검증된 버전 관리 시스템이다
- 로컬 저장, **commit**, **diff**, **restore**, **branch** 등 핵심 기능이 잘 갖춰져 있다
- **Git** 전체를 다시 구현하는 것은 비용이 너무 크다
- 우리의 핵심 가치는 **Git** 엔진 재구현이 아니라 **UI/UX** 추상화에 있다

즉:

- 사용자에게는 **Git**을 숨기고
- 내부적으로는 **Git**을 활용하는 구조가 가장 적합하다

6.2 **Git**이 **command** 창에서 동작하는 이유

터미널에서 ``git add``, ``git commit`` 같은 명령어를 사용할 수 있는 이유는 **Git**이 설치된 실행 가능한 프로그램이기 때문이다.

즉:

- ``git``은 단순한 문법이 아니라
- 사용자의 컴퓨터에 설치된 프로그램(binary)이다

그래서 앱도 내부적으로 **Git** 프로그램을 실행할 수 있다.

6.3 앱에서 **Git**을 사용하는 방법

앱이 **Git**을 활용하는 방법은 크게 두 가지가 있다.

방식 1. CLI(command) 실행

앱이 내부적으로 `git add`, `git commit`, `git push` 같은 명령을 실행

방식 2. Library 사용

Git 관련 기능을 함수 형태로 호출할 수 있는 라이브러리를 사용하는 방식

여기서 **library**는:

- > Git 명령어를 문자열로 실행하는 대신,
- > 코드 안에서 Git 기능을 함수처럼 호출할 수 있게 해주는 도구

라는 의미로 이해할 수 있다.

7. Git 설치 문제에 대한 논의

우리가 논의한 중요한 현실적 문제 중 하나는:

- > 사용자의 컴퓨터에 Git이 설치되어 있지 않으면 어떻게 할 것인가

현재 가장 현실적인 방향은 다음과 같다.

Git Dependency Strategy

1. 앱 실행 시 Git 설치 여부 확인
2. 설치되어 있으면 그대로 사용
3. 설치되어 있지 않으면 설치를 안내
4. 설치 완료 후 앱 기능 활성화
5. 이후에는 사용자가 Git을 직접 의식하지 않게 함

이 방식의 장점은 다음과 같다.

- 구현 난이도가 낮다
- Git의 안정성을 활용할 수 있다
- 사용자에게는 Git command를 숨길 수 있다

8. 우리가 조심해야 할 점

8.1 Git-less라는 표현의 정확성

내부적으로 Git을 사용한다면 “완전한 **Git-less**”라고 하기는 어렵다.

더 정확한 표현은 다음과 같다.

- Git abstraction layer
- Git UI layer
- beginner-friendly Git/GitHub manager

8.2 Snapshot과 Git commit을 동시에 핵심 구조로 두는 문제

둘 다 메인 기록 구조가 되면 **source of truth**가 두 개가 된다.

그러면 다음 문제가 생긴다.

- 복구 기준이 모호해짐
- 저장 구조가 복잡해짐
- 사용자가 어떤 기준으로 상태를 이해해야 하는지 혼란스러워짐

그래서 현재 방향에서는

****Git을 메인 엔진으로 두고, UI만 추상화하는 방식****이 더 자연스럽다.

8.3 제품 포지셔닝 혼란

우리는 아래 세 가지를 동시에 만들 수는 없다.

- Git 대체 시스템

- 로컬 안전 복구 시스템

- 쉬운 Git/GitHub UI 툴

현재 가장 적절한 선택은 세 번째에 가깝다.

즉:

> 이 프로젝트는 Git replacement가 아니라

> Git/GitHub UX abstraction product이다.

9. 현재 기준의 제품 정의

현재 논의를 바탕으로 하면,
이 프로젝트는 다음과 같이 정리할 수 있다.

> 비개발자와 초보 사용자도 **command** 창에서 Git 명령어를 직접 입력하지 않고,

> UI 기반 앱을 통해 로컬 프로젝트를 관리하고 GitHub와 쉽게 연결할 수 있도록 돕는 사용자 친화적 개발 관리 도구

더 짧게 쓰면:

> 로컬 Git과 GitHub workflow를 직관적인 UI로 추상화한 개발 관리 도구

10. 핵심 UX 방향

사용자에게는 Git 개념 대신 아래와 같은 표현을 제공하는 것이 맞다.

|Git 개념|사용자 표현|

|---|---|

|git add|변경 선택|

|git commit|저장하기|

|git push|GitHub에 업로드|

|branch|새 버전 만들기 / 실험 버전|

|checkout / restore|이전 상태로 돌아가기|

핵심은 사용자가 Git을 이해하는 것이 아니라,

****파일 작업 흐름만 이해해도 쓸 수 있어야 한다는 것****이다.

11. 현재 시점의 잠정 결론

지금까지의 브레인스토밍을 종합하면, 현재 가장 현실적이고 적절한 방향은 다음과 같다.

잠정 결론

- Git은 내부 엔진으로 사용한다
- 사용자는 **Git command**를 직접 다루지 않는다
- 앱은 **Git workflow**를 **UI workflow**로 바꾼다
- **Git** 설치 여부를 확인하고, 미설치 시 설치를 안내한다
- 핵심 가치는 **Git**을 없애는 것이 아니라 **Git/GitHub** 진입장벽을 낮추는 것이다

12. 앞으로 추가로 정리해야 할 것

다음 단계에서 팀 차원에서 더 구체화해야 할 항목은 다음과 같다.

1. 정확한 타겟 유저 정의
2. drag and drop 이후의 UI 흐름
3. 저장 / 업로드 / 복구의 사용자 언어 정리
4. GitHub 연동 방식
5. restore UX와 에러 처리 방식
6. 오픈소스 단계에서의 **Git** 설치 및 설정 정책

3/17 (화) git/github 구현시 발생가능
문제점

UI로 **Git workflow**를 많이 단순화할 수는 있지만, **merge conflict** 같은 문제를 “완전히 없애는 것”은 어렵다.

왜냐하면 그건 **UI**의 문제가 아니라, 동시에 같은 파일/같은 부분을 다르게 수정했을 때 생기는 버전 관리의 본질적 문제이기 때문이다. **Git**도 이런 경우 자동으로 항상 해결하지 못하고, 충돌이 나면 사용자가 직접 결정해야 한다.

그래서 이 프로젝트를 개발할 때의 핵심은:

충돌을 완전히 제거하는 것이 아니라

충돌이 덜 생기게 설계하고, 생겼을 때 사용자가 이해할 수 있게 **UI**로 안내하는 것이다.

아래처럼 보면 된다.

1. 가장 큰 구현 문제: **Git**의 복잡성을 어디까지 숨길 수 있는가

처음에는 “**Git command**만 안 보이면 쉬워질 것” 같지만, 실제 구현에 들어가면 문제가 더 많다.

예를 들면 사용자는 단순히 이렇게 생각한다.

- 저장하기
- 업로드하기
- 최신 버전 받기
- 이전 상태로 돌아가기

그런데 내부에서는 다음을 처리해야 한다.

- 현재 **repo**가 **Git repo**인지 확인
- **remote**가 연결되어 있는지 확인
- **branch** 상태 확인
- **uncommitted change** 확인
- **push** 가능한 상태인지 확인

- pull 했을 때 **fast-forward**인지, **merge**가 필요한지 확인
- 충돌 발생 여부 확인
- 인증 상태 확인

즉,
UI는 단순해야 하지만 내부 상태 머신은 꽤 복잡해진다.

2. merge conflict는 UI로 “없앨 수” 있는가?

완전히 없앨 수는 없다.

Git 공식 문서에서도, 같은 파일의 같은 부분이 서로 다르게 수정되면 Git이 **clean merge**를 못하고 충돌 상태로 멈출 수 있다고 설명한다. 이 경우 사용자가 충돌을 해결한 뒤 **merge**를 계속해야 한다.

즉:

- 충돌 자체를 **0**으로 만드는 것은 불가능
- 하지만 충돌이 자주 일어나지 않게 **UX**를 설계하는 것은 가능

예를 들면 앱에서 이런 식으로 설계할 수 있다.

가능한 방식

1. **push** 전에 먼저 **remote** 상태 확인
 - “업로드하기” 누르기 전에
 - 원격 저장소가 더 앞서 있는지 검사
 - **remote**가 앞서 있으면 바로 **push**하지 않고 “먼저 최신 변경을 받아야 합니다”라고 안내
2. 자동 **fast-forward**만 허용
 - **merge**가 필요 없는 경우만 자동 진행

- 복잡한 **merge** 상황이면 자동 업로드 중단
3. 초보자 모드에서는 **branch**를 거의 숨김
- **main** 하나만 쓰게 하고
 - “실험 버전 만들기” 정도로 제한
4. 충돌 위험 사전 경고
- 같은 파일이 **remote**에서도 수정되었으면
 - “이 파일은 다른 곳에서도 변경되었습니다” 표시

즉,

UI로 **conflict**를 예방하는 건 가능하지만, **conflict** 자체를 논리적으로 제거하는 건 어렵다.

3. 충돌이 났을 때 **UI**로 정확히 처리할 수 있나?

가능은 하지만, 어느 수준까지 지원할지 결정해야 한다.

레벨 1: 가장 현실적인 방식

앱이 충돌을 감지하면 단순히 안내만 하는 방식

예:

- “자동 병합이 불가능합니다”
- “아래 파일에서 충돌이 발생했습니다”
- “VS Code에서 열기”
- “내 버전 유지 / 최신 버전 보기 / 수동 수정”

이건 현실적이다.

레벨 2: 간단한 **conflict resolver UI** 제공

앱 안에서 3-way diff UI를 보여주는 방식

예:

- Base
- Local
- Remote

그리고 버튼:

- 현재 버전 사용
- 원격 버전 사용
- 둘 다 반영
- 직접 편집

이론상 가능하다. `libgit2` 같은 라이브러리도 `merge` 후 `index`를 검사하고 `conflict`를 해결하도록 설계되어 있다. 하지만 충돌이 생기면 결국 `caller`가 직접 `resolve` 과정을 처리해야 한다.

문제는 이 단계부터 UI 난이도가 급격히 올라간다는 점이다.

레벨 3: 완전 자동 충돌 해결

이건 위험하다.

왜냐하면 자동으로 선택하면:

- 내 코드가 사라질 수 있고
- 상대 코드가 사라질 수 있고
- 앱이 망가질 수 있고
- 사용자는 왜 이렇게 됐는지 이해 못 할 수 있다

Git도 `ours`, `theirs` 같은 전략을 줄 수는 있지만, 그건 “정답”이 아니라 “한쪽을 포기하는 정책”에 가깝다.

그래서 초보자용 제품이라면 오히려:

- 자동 해결을 남발하지 말고
- 안전하게 멈추고 설명하는 것

이 더 중요하다.

4. 실제 개발 시 자주 부딪히는 문제들

이 프로젝트는 merge conflict 말고도 구현 난점이 꽤 많다.

4.1 Git 설치/환경 차이

OS마다 다르다.

- Windows
- macOS
- Linux

Git 설치 위치도 다를 수 있고,
PATH 설정도 다를 수 있다.

그래서 앱 시작 시:

- git binary 존재 확인
- 버전 확인
- 최소 지원 버전 검사
- 없으면 설치 가이드 표시

이 단계가 필요하다.

4.2 인증 문제

GitHub push/pull은 인증이 필요하다.

GitHub는 비밀번호 방식 대신 **PAT**, **SSH**, **credential helper** 같은 방식을 사용한다. GitHub 공식 문서도 **PAT** 또는 **credential helper** 사용을 안내하고 있고, **Git Credential Manager**는 보안 저장과 브라우저 로그인 흐름을 제공한다.

문제는 사용자 입장에서는:

- **PAT**가 뭔지 모름
- **SSH key**가 뭔지 모름
- 인증 실패 메시지가 이해 안 됨

즉 **UI** 앱은 인증 **UX**도 직접 설계해야 한다.

예:

- “GitHub 로그인 연결”
- 브라우저 로그인 유도
- **credential manager** 감지
- 인증 실패 시 쉬운 문장으로 재안내

이 부분이 생각보다 중요하다.

4.3 Git 상태가 깨진 **repo** 처리

실제 사용자 **repo**는 항상 깨끗하지 않다.

예:

- **merge** 도중 멈춘 상태
- **rebase** 도중 중단된 상태
- **detached HEAD**
- **untracked** 파일이 너무 많음

- submodule 있음
- large file 있음
- permission 문제 있음

초보자용 앱이라면 이런 상태를 그냥 노출하면 안 되고,
“현재 프로젝트 상태를 자동 진단”하는 레이어가 필요하다.

즉 앱은 단순 버튼 앱이 아니라
사실상 **Git** 상태 해석기가 되어야 한다.

4.4 파일 삭제 / 이동 / 이름 변경 처리

사용자는 그냥 폴더에서 파일 이름 바꾸고 옮긴다.

그런데 **Git** 관점에서는:

- rename detection
- delete/add 처리
- tracked/untracked 구분

이런 게 들어간다.

UI에서는 그냥 “파일 이름 바뀜”처럼 보여야 하지만,
내부적으로는 **diff** 해석이 필요하다.

4.5 복구 UX

“이전 상태로 돌아가기”는 말은 쉽지만,
실제로는 여러 의미가 있다.

- 마지막 저장 취소
- 특정 파일만 되돌리기
- 전체 프로젝트를 이전 **commit**으로 되돌리기

- **remote** 기준으로 덮어쓰기
- 작업 중인 변경은 유지하면서 일부만 복구

이건 전부 **Git**에서 다른 명령/개념이다.

그래서 **UI**에서 “복구” 버튼 하나만 두면 안 되고,
복구의 범위를 분리해야 한다.

예:

- 파일 되돌리기
- 저장 전 변경 취소
- 이전 버전 열기
- 이 시점으로 프로젝트 복원

5. 초보자용 제품에서 특히 위험한 부분

이 프로젝트에서 제일 위험한 건
사용자를 편하게 해주려다가 너무 많은 걸 자동으로 처리하는 것이다.

예를 들어:

- 자동 merge
- 자동 overwrite
- 자동 pull 후 자동 push
- 자동 branch 전환
- 자동 conflict resolution

이런 건 데모에서는 좋아 보이지만 실제 사용에서는 사고가 난다.

초보자용 애플리케이션의 원칙은 이쪽이 더 맞다.

추천 원칙

- 안전하지 않으면 자동 실행하지 않기
- destructive action은 항상 미리보기 제공
- conflict는 멈추고 설명하기
- source of truth는 Git 하나로 유지하기
- 사용자가 “무슨 일이 일어났는지” 이해할 수 있게 하기

6. 그래서 MVP에서는 어디까지 해야 하나?

MVP에서는 merge를 “예쁘게 다 해결하는 앱”까지 가려고 하면 너무 무겁다.

더 현실적인 MVP는 이렇다.

MVP 권장 범위

- 로컬 폴더 열기
- Git repo 여부 확인
- 변경 파일 목록 보여주기
- 선택 저장(commit)
- GitHub 업로드(push)
- 최신 버전 받기(pull) 전 상태 검사
- 충돌 발생 시 감지 + 쉬운 안내
- 문제 파일 목록 표시
- 외부 에디터로 열기

MVP에서 과감히 뒤로 미뤄도 되는 것

- 앱 내부 full 3-way merge editor
- 복잡한 branch 시각화
- rebase UI
- cherry-pick UI
- stash UI
- submodule 지원
- advanced Git recovery

즉 처음부터 “Git 전체를 UI로 옮기기”가 아니라
가장 많이 쓰는 **happy path**를 매끄럽게 만들고, 예외는 안전하게 처리하는 게 맞다.

7. 제품적으로 가장 중요한 판단

이 프로젝트의 본질은:

Git을 단순화하는 앱이지

Git의 모든 복잡성을 없애는 앱은 아니다.

그래서 팀 내부에서 이 기준을 명확히 잡아야 한다.

좋은 방향

- 80%의 일반 흐름을 아주 쉽게 만든다
- 나머지 20%의 복잡한 상황은 감지해서 설명한다

위험한 방향

- 100% 상황을 UI 한 화면에서 다 처리하려고 한다

후자가 되면 결국 **Git client** 전체를 새로 만드는 수준이 된다.

8. 한 줄로 정리하면

merge conflict 같은 문제는 **UI**로 “예방”하고 “쉽게 설명”할 수는 있지만, 완전히 제거하거나 항상 자동 해결하는 것은 어렵다.

그래서 이 프로젝트의 성공 포인트는 **Git**을 숨기는 것 자체가 아니라, **Git**의 위험한 순간을 사용자가 덜 무섭게 느끼도록 설계하는 것이다.

3/17 (화) git 에서 자주 발생하는 문제점

가장 자주 발생하는 문제” + “코드 유실 0 원칙”으로 설계해야 한다.

핵심 원칙부터 먼저 잡으면 이거다.

절대 기준: 사용자의 코드가 날아가면 안 된다

이 프로젝트에서 가장 중요한 건 기능 수가 아니라 안전 규칙이다.

반드시 이렇게 가야 한다.

1. 파괴적 작업은 자동 실행 금지

- **overwrite**
- **hard reset**
- **force push**
- 자동 **conflict** 해결
- 자동 **clean**

2. 모든 중요한 작업 전 자동 백업 지점 생성

- 작업 직전 **lightweight backup commit** 또는 **stash/patch** 저장
- 실패 시 바로 복구 가능하도록

3. 미리보기 없는 반영 금지

- 바뀌는 파일 목록
- 삭제 파일
- 덮어쓰기 파일
- 충돌 파일
를 먼저 보여준 뒤 실행

4. **Git**을 메인 **source of truth**로 유지

- 별도 **snapshot** 시스템을 메인으로 두지 말고

- 필요하면 보조 안전장치로만 사용

Git 자체도 로컬 변경 임시 보관을 위해 `git stash`를 제공하고, 복구/되돌리기 계열 명령은 `reset`, `restore`, `revert`처럼 목적이 다르다. 즉 앱도 “되돌리기”를 하나의 버튼으로 뭉개지 말고 안전하게 분리해야 한다.

가장 자주 발생하는 문제 **1**: 저장 안 된 변경이 있는데 **pull/push/restore**를 누르는 경우

이게 진짜 흔하다.

사용자는 그냥:

- 파일 수정
- 아직 저장(commit) 안 함
- 최신 버전 받기 클릭
- 또는 이전 상태 복구 클릭

이런 흐름을 탄다.

그런데 이 상태에서 잘못 처리하면 로컬 변경이 섞이거나 덮어써질 수 있다.

해결

앱에서 어떤 동작이든 실행 전에 항상 작업 상태를 검사해야 한다.

필수 체크

- tracked modified 파일
- staged 파일
- untracked 파일
- 삭제된 파일

Git은 **untracked** 파일도 따로 보여주며, 기본적으로 새 파일은 사용자가 명시적으로 추가하기 전까지 **commit snapshot**에 포함되지 않는다. 그래서 “저장된 줄 알았는데 안 올라감” 문제가 자주 생긴다.

UI 정책

- 변경이 있으면 바로 진행하지 말고 중단
- 다음 셋 중 하나를 선택하게 하기
 - 지금 변경 저장하기
 - 임시 보관하기
 - 취소하기

추천 구현

- pull 전: auto stash 또는 temp safety commit
- restore 전: patch 파일 백업 + 확인창
- push 전: untracked 파일 포함 여부 명확히 표시

가장 자주 발생하는 문제 2: non-fast-forward push 거절

이것도 엄청 흔하다.

상황:

- 내 로컬보다 GitHub 쪽이 더 최신
- 사용자가 그냥 업로드 클릭
- push 실패

GitHub 공식 문서도 remote가 앞서 있으면 **non-fast-forward**로 push가 거절되고, 먼저 upstream 변경을 가져와 통합해야 한다고 설명한다.

해결

앱에서 “업로드” 버튼을 누르면 바로 **push**하면 안 된다.

올바른 순서

1. **fetch**
2. **local vs remote** 비교
3. **fast-forward** 가능 여부 확인
4. 가능하면 진행
5. 불가능하면 “먼저 최신 변경 확인 필요” 안내

UI 문구 예시

- “GitHub에 더 최신 변경이 있습니다.”
- “바로 업로드하면 충돌 위험이 있습니다.”
- “최신 변경 확인 후 이어서 업로드하세요.”

절대 금지

- 자동 **force push**
- 자동 **overwrite push**

이건 초보자용 제품에서 가장 위험하다.

가장 자주 발생하는 문제 **3: merge conflict**

이건 완전 제거는 못 한다.

같은 파일의 같은 부분을 서로 다르게 바꾸면 Git도 자동 병합을 못 하고 멈춘다. Git 공식 문서도 충돌이 자동 해결되지 않으면 **merge**가 중단되고, 사용자가 **merge --abort** 또는 해결 후 **merge --continue**를 해야 한다고 설명한다.

해결

목표는 “자동 해결”이 아니라 “안전하게 멈추기”다.

추천 정책

- 기본 모드에서 복잡한 **auto merge** 금지
- **conflict** 발생 시 즉시 중단
- 충돌 파일 목록 표시
- 현재 상태 백업 보존
- 해결 옵션 제공:
 - 내 변경 보기
 - 최신 버전 보기
 - 외부 에디터에서 열기
 - 병합 취소

MVP에서는

앱 안에 **full merge editor**까지 안 넣어도 된다.

대신:

- 충돌 감지
 - 문제 파일 강조
 - “병합 취소”
 - “VS Code로 열기”
정도만 있어도 충분히 현실적이다.
-

가장 자주 발생하는 문제 **4**: 사용자가 새 파일을 만들었는데 저장 안 된 경우

초보자는 이걸 많이 헛갈린다.

예:

- 새 파일 생성
- 기존 파일만 수정한 줄 알고 저장 클릭
- 나중에 보니 새 파일이 **Git**에 안 들어감

Git은 **untracked** 파일을 따로 관리하고, 사용자가 명시적으로 **add**하지 않으면 **commit**에 포함되지 않는다.

해결

UI에서 “변경 파일”과 “새 파일”을 분리해서 보여줘야 한다.

예시

- 수정된 파일
- 새로 만든 파일
- 삭제된 파일

그리고 저장하기 전에:

- 새 파일 포함 여부
- 삭제 반영 여부
를 체크박스로 보여주면 된다.

가장 자주 발생하는 문제 **5**: **restore / reset** 개념 혼란

사용자는 “되돌리기”를 원하지만, 실제로는 여러 종류가 있다.

- 파일만 되돌리기
- 마지막 저장 전 상태로 되돌리기
- 특정 시점으로 프로젝트 복원
- 원격 기준으로 다시 맞추기

Git도 `reset` / `restore` / `revert`를 구분한다. `revert`는 새 `commit`으로 되돌리고, `restore`는 `working tree`나 `index`에서 파일을 복원하며, `reset`은 더 강한 상태 이동이다.

해결

UI에서 “되돌리기” 하나로 뭉치지 말아야 한다.

추천 분리

- 파일 변경 취소
- 마지막 저장 취소
- 이전 버전 보기
- 이 시점으로 복원

절대 금지

- 사용자가 의미를 모르는 상태에서 `hard reset` 실행

가장 자주 발생하는 문제 6: 인증 실패

실사용에서는 이것도 자주 나온다.

예:

- GitHub 로그인 만료
- 토큰 권한 부족

- credential manager 미설정
- workflow 파일 push 권한 문제

해결

에러를 Git 원문 그대로 보여주면 안 된다.

UI에서 해야 할 일

- “인증이 완료되었습니다”
- “다시 로그인 필요”
- “이 저장소에 업로드 권한이 없습니다”
- “추가 권한이 필요한 파일이 포함되어 있습니다”

처럼 번역해서 보여줘야 한다.

가장 자주 발생하는 문제 7: 큰 저장소 / 너무 많은 파일 때문에 느려짐

실제로 repo가 커지면 status, diff, untracked scan도 느려질 수 있다. Git 문서도 untracked 파일 탐색이 큰 working tree에서 시간이 걸릴 수 있다고 설명한다.

해결

- 백그라운드 status 갱신
- 전체 스캔 debounce
- 큰 폴더는 점진적으로 로딩
- .gitignore 추천
- node_modules, build, dist 같은 대형 생성 폴더 경고

UI 문구 예시

- “생성 파일이 많아 프로젝트 확인에 시간이 걸리고 있습니다.”
- “이 폴더를 제외하면 더 빨라집니다.”

가장 자주 발생하는 문제 **8**: 사용자가 병합 중/리베이스 중인 **repo**를 다시 여는 경우

현실에서 **repo**는 항상 깨끗하지 않다.

예:

- merge 도중 멈춤
- conflict 해결 전 종료
- rebase 중단
- detached HEAD

해결

앱 시작 시 **repo health check**가 필요하다.

시작 시 검사할 것

- merge in progress
- rebase in progress
- detached HEAD
- local changes
- untracked changes
- remote ahead/behind

UI

- “이 프로젝트는 병합 도중 멈춘 상태입니다.”
 - “먼저 이어서 해결하거나 취소해야 합니다.”
-

가장 중요한 설계 원칙: **Code Loss Prevention Layer**

네가 말한 “절대 코드가 날아가면 안 된다”를 구현 수준으로 바꾸면 이렇게 된다.

반드시 넣어야 하는 안전장치

1. 작업 전 안전 백업

모든 위험 작업 전:

- stash 또는
- patch export 또는
- hidden backup branch / temp commit

중 최소 하나 자동 생성

2. 파괴 작업 2단계 확인

예:

- 되돌리기
- 덮어쓰기
- 병합 취소
- 삭제 반영

은 바로 실행 금지

3. force 계열 기본 비활성화

- force push
- hard reset
- clean -fd
- branch delete 강제 옵션

기본 UI에서 숨기기

4. 복구 버튼 제공

앱 내에 항상 있어야 한다.

- “마지막 안전 지점 복원”
- “임시 보관 변경 복원”
- “방금 전 작업 되돌리기”

5. 로그 남기기

사용자 눈에 보이게:

- 언제
 - 어떤 작업을
 - 어떤 파일에
 - 어떤 결과로
적용했는지 기록
-

MVP에서 꼭 지원해야 할 문제와 해결만 추리면

MVP 필수

- 변경 파일/새 파일/삭제 파일 구분
- push 전 remote 상태 확인
- pull 전 local change 확인
- conflict 감지 후 중단
- 위험 작업 전 자동 백업
- restore 전 미리보기
- 인증 실패 메시지 번역
- 마지막 작업 복구 기능

나중으로 미뤄도 되는 것

- 앱 내부 full merge editor
- 복잡한 branch 시각화
- interactive rebase
- cherry-pick UI
- submodule 완전 지원

사용자에게 안전을 강요하는 게 아니라,

앱이 뒤에서 안전장치를 자동으로 깔아주는 구조가 되어야 한다.

사용자가 이렇게 느껴야지 직관적이다.

- 눌러도 큰일이 안나는것
- 실수해도 복구가 되는것
- 뭔가 잘못되면 앱이 알려주는것

3/18 (수) / 3/21 (금) 미팅

발표 순서 정하기

1. 프로젝트 변경 이유 – what you said, we completely understand the barrier that we've met and discussed. (DataBase, 토큰소모): → next-gen git (깃헙 사용 힘든것같은데 쉽게 이런 보조툴을 만드는건 어떨까?, 깃에 대한 두려움?) (윤하)
2. 시장성 조사 / 차별점 : 우리겐 뭐가 다를까 (관호)
3. 타겟 / 개발 방향 (종훈)
 - ↳ 타겟 : 1) 개발자지만 git이 익숙하지 않은 사람 -> 팀 플젝을 할 때 github을 사용할 텐데 그럴 때
Git command를 사용해야 하는 부담감을 줄여줌
 - 2) 비개발자지만 바이브 코딩하고 싶은 사람 -> 이 사람들은 github를 사용할 필요 X / 왜 우리의 앱을 쓰면 좋은가? -> 바이브 코딩하다가 마음에 안 드는 부분들이 있으면 이전 버전으로 되돌릴 수 있음 / 그래서 이 사람들은 github에 올리는 게 목적이 아니라 로컬에서 git만 사용할 수 있도록 하면 좋을듯!
 - ↳ 개발 방향: 로컬에서 사용할 수 있는 command들을 먼저 backend적으로 구현하고 나중에 github 연동하여 사용할 수 있는 command들을 backend적으로 구현하면 좋을듯.
- UI는 backend로 구현이 되는지 확인하고 그 이후에 하면 좋을듯
4. UI – Figma w/ MVP (찬영)
5. Backend 데모 보여주기 (찬영)
6. We'd like to finalize with this project setup. Advise? Add-ons?

1. Usability: how to make it easier? Think about itit's valuable to think /UX, git
2. AI component: to add cool features, ask AI based on our Git repo
Curate the way or info that makes it more useful for them
3. API token usage - feel free to use as much as you can.

Take the info, curate it, and use LLM to sort suggest things for ppl to do w/ it. → interface layer

Git 별칭(Alias) 설명

자주 사용하는 Git 명령어 단축 별칭 모음

별칭	원래 명령어	설명	자연어
 기본 상태 확인			
g	git	git 단축어	
gs	git status -sb	변경된 파일 목록을 간결하게 표시 (-s 짧게, -b 브랜치 표시)	변경된 파일이 뭔지 나열하는 느낌 1. 지금 있는 branch 2. 깃 애드한 파일들 3. 바꾸긴 했는데 깃 애드는 안했어 4.
 스테이징 (add)			
ga	git add	특정 파일 스테이징 (ga file.txt)	파일 고르기 (보낼 사진 한장 선택)
gaa	git add -A	모든 변경사항 한 번에 스테이징	전부 고르기 (앨범에서 전체 선택) c
ga.		모든 파일 한 번에 스테이징	
 커밋 (commit)			
gc	git commit	커밋 (에디터 열림)	저장해
gcm	git commit -m	메시지를 바로 입력해서 커밋 (gcm "메시지")	저장해 + 메모 남기기 → 결국 두 가지는 다른데 메모 유무 차이.
 브랜치 (branch / checkout)			
gch	git checkout	브랜치 전환	그 작업 공간으로 이동해
gchb	git checkout -b	새 브랜치 만들면서 전환	새 작업 공간 만들고 거기서 이동해
gb	git branch	브랜치 목록 확인	지금 작업 공간이 몇 개야?
gbd	git branch -d	브랜치 삭제	이 작업공간은 이제 필요 없으니깐 지워
 원격 저장소 동기화 (pull / push)			
gp	git pull --ff-only → git fetch + git merge	fast-forward만 허용하는 pull (충돌 방지)	서버에 새 내용 있으면 (충돌 위험 없을 때만) 로컬로 가져와
gpr	git pull --rebase → git fetch + git rebase	rebase 방식으로 pull	서버 내용 가져오면서 내 작업을 그 위에 얹어 → 깃 풀이랑 똑같은데 메인에 수정됐으니깐 바뀐 메인을 땡겨오고 내가 수정한 내용을 위에 올려서 정리해줘

gpsh	git push	원격 저장소에 push	내 저장 기록 서버에 올려
 히스토리 / 변경 내용 확인			
gl	git log --oneline --graph --decorate	커밋 히스토리를 그래프로 간결하게 표시	
gsh	git show	특정 커밋의 변경 내용 확인	
gd	git diff	워킹 디렉토리의 변경사항 확인	
gds	git diff --staged	스테이징된 변경사항 확인	
 되돌리기 (restore)			
gr	git restore	워킹 디렉토리의 변경사항 되돌리기	이 파일 수정한거 없던걸로 해 → 전버전으로 돌아가줘
grs	git restore --staged	스테이징 취소 (⚠ em dash 오류 주의 → --staged로 수정 필요)	담아둔거 다시 꺼내
 임시 저장 / 작업 트리			
gst	git stash	현재 변경사항 임시 저장	저장이 되지만 기록이 안남 → 에러가 터지면 다른 것을 해야 하니까, 일단 임시 저장하고 깃을 이용해서 에러를 해결한 후에 다시 할 수 있게 하는 것. → 커밋할 준비가 안되서 임시저장
gstp	git stash pop	임시 저장한 내용 꺼내기	깃 스테시 한것을 다시 불러오는 역할 → 저장 안된 기록을 다시 복구 그래서 커밋을 할지 버릴지 결정할 수 있음
gwt	git worktree	여러 브랜치를 동시에 다른 폴더에서 작업	같은 저장소를 여러 폴더에서 동시에 열어서 작업해

⚠ 주의: grs 별칭의 —staged 대신에 em dash(—)로 되어 있으면 오류가 납니다. --staged (하이픈 두 개)로 수정하세요.

업무에서 많이 사용하는 git은 이정도 예요.

```
alias g='git'
alias gs='git status -sb'
alias ga='git add'
alias gaa='git add -A'
alias gc='git commit'
```

```
alias gcm='git commit -m'
alias gch='git checkout'
alias gchb='git checkout -b'
alias gb='git branch'
alias gbd='git branch -d'
alias gp='git pull --ff-only'
alias gpr='git pull --rebase'
alias gpsh='git push'
alias gl='git log --oneline --graph --decorate'
alias gsh='git show'
alias gd='git diff'
alias gds='git diff --staged'
alias gr='git restore'
alias grs='git restore --staged'
alias gst='git stash'
alias gstp='git stash pop'
alias gwt='git worktree'
```

보통 `.zshrc`에 `alias`로 추가해서 사용하고 있고, `git` 명령어를 모두 알필요는 없고 `git`이 꼬이거나 해서 풀어야 할 일이 생기면 그제서야 사실 새로운 거 배워도 충분합니다. 저기에다가 `git merge`, `git rebase`, `git revert` 정도 추가하면 거의 제가 사용하는 전부예요.

클로드는 내부적으로 사실 뭐든 다 사용하겠지만 그것도 저기서 크게 벗어나지 않을거고

3/30 (월) 미팅

Git 설치 버튼 -> window / mac

- git install
- git config --global user.name "Your Name"
- git config --global user.email "your@email.com"
- git config --list
- Git hub account 만들기 (pop up - 같은 이름/이메일?)

Git main branch 만들기

Git add / commit / push

Git log / diff / status

Git pull (clone)

Git restore -> 되돌리기

Git 설치 방법

1. Mac

✓ 방법 1 (가장 쉬움)

터미널 열고 그냥 입력:

`git --version`

👉 만약 안 깔려있으면

👉 자동으로 설치하라고 팝업 뜸 (Xcode Command Line Tools)

→ "Install" 누르면 끝

✓ 방법 2 (Homebrew 사용)

`brew install git`

👉 Homebrew 쓰는 경우 추천

2. Windows

👉 공식 사이트에서 다운로드:

- Git for Windows

<https://git-scm.com/install/windows>

설치할 때:

- 그냥 **Next** 계속 누르면 됨 (default settings 추천)

 설치 확인

`git --version`

👉 예: `git version 2.42.0`

⚙️ 처음 1회 설정 (중요 ★)

설치 후 꼭 해야됨: - 이걸 이름, 이메일 input 받으면 자동으로 아래 코드 입력되서 돌아가게

하면 될것 같습니다

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

👉 이걸 commit 기록에 남는 정보

🔍 설정 확인

```
git config --list
```

4/3 (금) Meeting w/ Honda

1. AI 활용

- a. Ppl do not know what they want -> AI can help vibe coders to know “what steps they need to go”, AI can give high level plan of the prompted project idea, Ryan liked the idea of “how do we make vibe coding easier?”, a good analogy would be auto completion with grammar assisting tools
- 2. Instead of a desktop application, a web app would be the best, a vscode extension will be great as well, how about cli tool!!!!!!!!????? -> email Ryan!!!!
 - a. Think about the vibe coders using phones or tablets to code, they need a new thing to do version control, web app would be great

4/8 (수) 팀미팅

1. AI 자동 저장 메시지 (Auto Save Message Generator) (찬영)

- 비개발자가 가장 귀찮아하고 어려워하는 것이 "저장할 때 뭐라고 적어야 하지?" (Commit Message 작성) 입니다.
- 작동 방식: 앱 내부적으로 git diff를 실행하여 변경된 파일 내용들을 수집한 뒤, 이를 AI API(Claude, OpenAI 등)로 보냅니다.
- 사용자 경험: 사용자가 [저장하기] 버튼을 누르면, AI가 "메인 페이지의 배경색을 파란색으로 변경하고 오타를 수정함"처럼 사람이 이해하기 쉬운 문장으로 변경 사항을 자동 요약해 줍니다. 사용자는 확인만 누르면 됩니다.

2. 자연어 기반 타임머신 (Natural Language "Undo") (윤하)

- 버전 복구(git log, git checkout, git reset)는 개발자에게도 두려운 영역입니다. 이 과정을 대화형으로 풀어냅니다.
- 작동 방식: 사용자가 검색창에 "어제 오후에 작업했던 상태로 돌려줘" 또는 "빨간색 버튼 지우기 전으로 돌아갈래"라고 입력합니다.
- 사용자 경험: AI가 로컬 Git 히스토리(시간, 메시지, 파일 변경 내역)를 분석하여 사용자가 원하는 정확한 시점을 찾아냅니다. "이 시점(어제 오후 3시: 메인 이미지 수정됨)으로 되돌릴까요?"라고 제안하고, 버튼 하나로 안전하게 복구(git restore 등)를 수행합니다.

3. (승연)

- Git install 확인하는 윈도우
- API 연결, 데스크탑 앱 polish, 깃헙 공유
- 스마트 충돌 번역기 (Smart Conflict Resolver) (승연)
 - 사용자가 클라우드(GitHub)에서 최신 버전을 가져올 때 발생하는 '충돌(Conflict)'은 이 앱의 가장 큰 위험 요소입니다. 코드 사이사이에 섞여 있는 <<<<<<< HEAD 같은 기호는 비개발자에게 공포 그 자체입니다.
 - 작동 방식: Git 충돌이 발생하면, 앱이 충돌 난 텍스트를 AI에게 보냅니다.
 - 사용자 경험: "클라우드에는 'A문장'이 있고, 내 컴퓨터에는 'B문장'이 있습니다. 어떻게 합칠까요?"라고 일상적인 언어로 번역해서 보여줍니다. 더 나아가 AI가 두 문장을 자연스럽게 합친 'AI 추천 병합본'을 제시할 수도 있습니다.

4. 주간 작업 리포트 자동 생성 (Changelog Generator) (관호)

- 프리랜서, 디자이너, 작가 등 비개발자 직군이 자신의 작업 이력을 증명하거나 정리할 때 매우 유용합니다.
- 작동 방식: 특정 기간 동안 쌓인 저장(Commit) 내역을 AI가 종합하여 읽기 좋은 마크다운 문서나 이메일 형태로 요약합니다.
- 사용자 경험: [이번 주 작업 요약보기] 버튼을 누르면, "이번 주에는 총 15번의 저장이 있었고, 주로 '포트폴리오 페이지 디자인 완성'과 '연락처 폼 오타 수정'에 집중하셨네요."라는 친절한 리포트를 제공합니다.

5. 토큰 사용량 시각화 (종훈) -> claude 가능 gpt는 api 사용시에만 가능

- 참고:
<https://github.com/HongChaeMin/token-garden-app/blob/main/docs/superpowers/plans/2026-03-13-token-garden.md>

4/16 (목) 미팅

1. Demo
 - a. Make a new file in VS Code and commit/push using our GAT app
 - b. Natural language undo feature - 윤하
 - w/ AI API restore two days ago, restore before “commit message”
 - w/ AI API – file insight, connected files
 - w/ AI API – Review untracked files by analyzing the dummy/temporary files to easily detect and delete them
2. Explain AI features we’re going to implement:
 - a. Auto Save Message Generator - 찬영
 - b. Weekly Report & Change Log - 재운
 - c. Things to polish (AI API Key Settings, GitHub Connection) 만드었는데 안된부분-종훈
3. Questions
 - a. Are we also writing blog?
 - b. Graphify opensource
<https://github.com/safishamsi/graphify?tab=MIT-1-ov-file#readme>

Ryan’s Feedback

- 1.

4/17 (금) Ryan 미팅

1. 새로운 개발 패러다임으로의 포지셔닝

라이언은 현재 팀이 구축한 기능들이 전통적인 소프트웨어 개발 방식(커밋 메시지 작성 등)에 기반하고 있다는 점을 짚으며, *****바이브 코딩(vibe coding)*****과 같은 새로운 패러다임에 맞춘 포지셔닝을 제안했습니다

- 버전 관리의 자동화: 사용자가 버전 관리에 대해 전혀 모르더라도 배경에서 버전 관리가 자연스럽게 이루어지는 환경을 지향할 것을 조언했습니다
- 차별화 전략: "왜 깃허브 **GUI**(GitHub GUI)를 쓰지 않는가?"라는 질문에 대비해야 합니다. 라이언은 리포트 생성, 작업 시각화, 향후 할 일 추천 등 깃허브 **GUI**가 제공하지 못하는 **AI** 기반 기능을 강조하라고 조언했습니다
- 시장성: 이 프로젝트를 *****비전문가나 입문자를 위한 버전 관리 도구*****로 정의하면 훨씬 더 시장성 있는 이야기가 될 것이라고 언급했습니다

2. 스토리텔링과 피칭(Pitch)

프로젝트를 단순히 '데스크톱 앱을 만들었다'고 설명하는 것은 성과를 과소평가하는 것이라고 경고하며, 강력한 스토리를 만들 것을 주문했습니다

- 핵심 **AI** 기능 강조: 여러 기능 중 하나(예: 자동 리포트 생성)를 핵심으로 잡아 이를 중심으로 전체 프로젝트를 피칭하는 것이 좋습니다
- 의미 있는 데이터 제공: 단순한 커밋 횟수가 아니라, **LLM**을 활용해 작업의 의미적 설명이나 난이도 점수 등을 산출하여 생산성을 보여주는 도구로서의 가능성을 제시했습니다

3. 커리어 발전을 위한 팀 블로그 작성

라이언은 이번 캡스톤 프로젝트가 팀원들의 취업이나 진학에 실질적인 자산이 되기를 바라며, 팀 블로그 작성을 강력히 권고했습니다

- 블로그 포함 내용:
 1. 팀원 소개 및 링크드인 프로필
 2. 해결하려 했던 문제 정의와 기술적 솔루션
 3. 학습 과정의 강조: 단순히 무엇을 아는지가 아니라, 모르는 것을 어떻게 빠르게 배워서(**on the fly**) 적용했는지를 보여주는 것이 채용 담당자에게 매력적입니다
 4. 기술적 난관을 극복한 사례 및 향후 개선 방향

라이언은 이 블로그를 **99P Labs** 블로그에 게시하고 링크드인에 공유하여 팀원들을 홍보해주겠다고 약속했습니다

4. 향후 계획 및 지원

- 최종 마무리: 남은 **2주** 동안 프로젝트를 가능한 수준까지 완성하고 블로그를 작성하는 것으로 과제를 마무리했습니다

- 추천서 지원: 프로젝트를 성실히 마친 후, 대학원 진학이나 취업 시 ****추천서(letter of recommendation)****가 필요하면 기꺼이 작성해주겠다고 제안했습니다
- 네트워킹: 링크드인을 통해 지속적으로 연결될 것을 독려하며 실무적인 네트워킹의 중요성도 언급했습니다

4/20 (월) 미팅

고쳐야 할 부분

- **Git push / pull** 버튼
- 파일에 대한 설명이 때때로 바뀜

추가할 부분

- 오픈소스를 가져올 때 **url** 주소 넣으면 되도록
- **Git pull**할 때 내용 분석 및 요약해서 알려주기